

ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS

TrajCenter v2.0

MANUEL UTILISATEUR ET GUIDE D'INTÉGRATION

TrajCenter avec ABB Robot Web Services

Auteurs :

Josselin SCHUMACKER

Clément RACINET

16 septembre 2026



Avant-propos

Ce document présente l'utilisation de TrajCenter v2.0 pour la conversion, la gestion et le transfert de trajectoires industrielles vers un robot ABB RobotWare 6.x.

Il s'adresse principalement aux utilisateurs souhaitant :

- convertir des trajectoires au format TrajCenter ;
- utiliser l'interface terminal ou la ligne de commande ;
- lancer la supervision Python via ABB Robot Web Services ;
- exploiter les trajectoires chargées depuis un programme RAPID.

TrajCenter v2.0 repose exclusivement sur ABB Robot Web Services. L'ancien protocole TCP de TrajCenter v1 n'est plus utilisé.

Table des matières

Introduction	5
1 Démarrage rapide	5
1.1 Conditions préalables	5
1.2 Étape 1 – Installer TrajCenter	6
1.3 Étape 2 – Créer le fichier de configuration	6
1.4 Étape 3 – Préparer la configuration RAPID	7
1.5 Étape 4 – Convertir une trajectoire	8
1.6 Étape 5 – Vérifier le contenu du store	8
1.7 Étape 6 – Démarrer le superviseur	8
1.8 Étape 7 – Demander la trajectoire depuis RAPID	9
1.9 Étape 8 – Vérifier avant l’exécution	10
1.10 Résumé du premier cycle	10
2 Installation et utilisation côté PC	11
2.1 Installation	11
2.1.1 Prérequis	11
2.1.2 Organisation des répertoires utilisateur	11
2.1.3 Choix de l’environnement Pixi	12
2.1.4 Installation d’un poste de préparation	12
2.1.5 Installation d’un PC connecté au robot	13
2.1.6 Installations spécialisées	13
2.1.7 Vérification de l’installation	13
2.2 Configuration du poste et du robot ciblé	14
2.2.1 Qui configure quoi ?	14
2.2.2 Priorité des paramètres	15
2.2.3 Paramètres configurables par l’utilisateur	15
2.2.4 Création du fichier de configuration	15
2.2.5 Paramètres qui ne doivent pas être placés dans le fichier <code>.env</code>	16
2.3 Conversion d’une trajectoire	17
2.3.1 Formats pris en charge	17
2.3.2 Conversion avec le CLI	17
2.3.3 Portabilité et compatibilité	18
2.4 Gestion du store local	18
2.4.1 Lister les trajectoires	18
2.4.2 Inspecter une trajectoire	19
2.5 Supervision d’un robot ABB	19
2.5.1 Prérequis de communication	19
2.5.2 Rôle du robot pendant le transfert	20
2.5.3 Lancement avec un fichier de configuration	20
2.5.4 Surcharges ponctuelles	20
2.5.5 Fonctionnement de la supervision	21
2.5.6 Journaux et codes de retour	21
2.6 Export d’une trajectoire	21
2.7 Interface terminal	22
2.7.1 Lancement de la TUI	22
2.7.2 Écran d’accueil	23

2.7.3	Menu principal	23
2.7.4	Navigation au clavier	24
2.7.5	Écran de conversion	25
2.7.6	Écran d'export	25
2.7.7	Écran du store local	26
2.7.8	Écran des paramètres	26
2.7.9	Écran de supervision robot	28
2.8	Référence de la ligne de commande	28
3	Utilisation côté RAPID	29
3.1	Installation et principe de fonctionnement	30
3.1.1	Installation du module sur le contrôleur	30
3.1.2	Principe d'une requête RAPID	30
3.1.3	Conditions nécessaires au fonctionnement	31
3.2	Configuration de la cellule	31
3.2.1	Initialisation de l'API	31
3.2.2	Déclaration des tools et des workobjects	32
3.2.3	Maintenance des tools et des workobjects	33
3.2.4	Configuration des valeurs par défaut	33
3.2.5	Routine de configuration recommandée	34
3.3	Chargement d'une trajectoire	35
3.3.1	Rafraîchissement des métadonnées	35
3.3.2	Sélection par nom	36
3.3.3	Sélection par index	36
3.3.4	Invalidation du payload précédent	36
3.3.5	Publication du nouveau payload	37
3.3.6	Attente de la fin du transfert	37
3.3.7	Contrôle du type de process chargé	37
3.3.8	États et diagnostic du transfert	38
3.4	Inspection des données chargées	39
3.4.1	Contenu d'un point	39
3.4.2	Bornes valides	39
3.4.3	Routines d'accès	40
3.4.4	Accès aux paramètres process	40
3.5	Exécution d'une trajectoire	41
3.5.1	Précautions avant l'exécution	41
3.5.2	Exécution avec l'API unifiée	42
3.5.3	Mouvements circulaires	43
3.5.4	Comportement des hooks process	44
3.6	Exemple complet d'intégration	44
3.7	Procédure d'utilisation courante	48
4	Diagnostic et résolution des erreurs	49
4.1	Méthode générale de diagnostic	49
4.2	Impossible de se connecter au contrôleur	50
4.3	Erreur d'authentification	50
4.4	Mastership refusée	51
4.5	Store local introuvable ou vide	52
4.6	Trajectoire inconnue	53

4.7	Tool ou workobject absent	54
4.8	Trajectoire trop grande	55
4.9	Requête RAPID restant à TRUE	56
4.10	Timeout RAPID	56
4.11	Superviseur arrêté	57
4.12	Module ou tâche RAPID incorrects	58
4.13	Autres erreurs de validation de trajectoire	59
4.14	Erreurs internes, de sérialisation ou de réponse RWS	60
4.15	Reprise après une erreur	61
5	Référence technique et intégration RAPID avancée	62
5.1	Architecture du module système	62
5.1.1	Rôle et contenu du module	62
5.1.2	Flux fonctionnel des données	63
5.1.3	Validité du payload chargé	66
5.1.4	Compatibilité des versions	67
5.2	API RAPID publique	67
5.3	Intégration des paramètres process	68
5.3.1	Catalogue des process exécutables	68
5.3.2	Accès contrôlé aux paramètres process	68
5.3.3	Implémentation d'un process métier	69
5.4	Routines internes et graphe d'appels	69
5.4.1	Principales routines de support	70
5.4.2	Moteur commun de mouvement	70
5.4.3	Graphe d'appels	71
5.5	Règles de modification du module	74
5.5.1	Modifications propres à la cellule	74
5.5.2	Ajout ou activation d'un process	74
5.5.3	Modifications du protocole	75

Introduction

TrajCenter v2.0 est un outil de préparation, de stockage et de transfert de trajectoires industrielles vers un robot ABB fonctionnant sous RobotWare 6.x (testé uniquement sur 6.08 et 6.15). Il permet de convertir plusieurs formats de fichiers en une archive commune portant l'extension `.trajcenter`, puis de transférer la trajectoire sélectionnée vers le contrôleur au moyen d'ABB Robot Web Services.

TrajCenter peut être utilisé selon deux modes complémentaires :

- **hors ligne**, sur un poste de préparation, pour convertir, consulter et exporter des trajectoires sans connexion à un robot ;
- **en ligne**, sur un PC connecté à la cellule, pour superviser les échanges avec le contrôleur ABB et charger une trajectoire dans le module RAPID `TRAJCENTER`.

La conversion reste indépendante du robot. Une archive `.trajcenter` peut donc être préparée, vérifiée et conservée sans accès au contrôleur. Lors du transfert, TrajCenter prend en compte les tools, les workobjects, les valeurs par défaut et les types de process exposés par la cellule. Les limites prises en charge par Python sont également vérifiées avant l'écriture, tandis que les dimensions du module RAPID constituent la limite finale côté contrôleur.

Le fonctionnement général repose sur quatre éléments :

- les fichiers sources, par exemple CSV, Excel, APT ou RAPID ;
- le store local contenant les archives `.trajcenter` ;
- le superviseur Python, chargé des échanges avec le contrôleur ;
- le module système RAPID `TRAJCENTER.sys`, chargé dans la tâche robot configurée.

La communication entre le PC et le contrôleur utilise exclusivement ABB Robot Web Services. L'ancien serveur TCP et le protocole de communication de TrajCenter v1 ne sont plus utilisés.

Le présent manuel s'adresse principalement aux utilisateurs, automaticiens et roboticiens chargés de préparer les trajectoires, de configurer la connexion, de démarrer la supervision et d'exploiter les données chargées depuis un programme RAPID. Les opérations de développement interne du logiciel ne sont abordées que lorsqu'elles sont nécessaires à l'intégration ou au diagnostic.

La section suivante présente un premier cycle d'utilisation complet. Les sections ultérieures détaillent l'installation, la configuration, les commandes disponibles, l'utilisation de l'interface terminal et l'intégration dans un programme RAPID.

1 Démarrage rapide

Cette section permet de réaliser un premier essai complet de TrajCenter : installation du logiciel, configuration de la connexion, conversion d'un fichier, démarrage du superviseur puis demande de chargement depuis RAPID.

La procédure utilise l'environnement `Pixi full`, qui contient à la fois les outils locaux, l'interface terminal et le client ABB Robot Web Services.

Dans ce manuel, le terme « store local » désigne le répertoire contenant les archives `.trajcenter`.

1.1 Conditions préalables

Avant de commencer, vérifier les points suivants :

- Pixi est installé sur le PC ;

- le dépôt TrajCenter est disponible localement ;
- le PC peut joindre le contrôleur ABB sur le réseau ;
- ABB Robot Web Services est accessible sur le contrôleur ;
- un compte RWS disposant des autorisations nécessaires est disponible ;
- le fichier `TRAJCENTER.sys` est chargé comme module système dans la tâche RAPID utilisée, généralement `T_ROB1` ;
- la configuration RAPID contient les tools et workobjects nécessaires à la trajectoire, ou des valeurs par défaut adaptées ;
- les premiers essais peuvent être réalisés dans RobotStudio ou dans des conditions de sécurité adaptées sur la cellule réelle.

Le chargement réussi d'une trajectoire ne garantit ni l'absence de collision, ni l'accessibilité des positions, ni la sécurité de son exécution. La trajectoire doit être vérifiée avant tout mouvement réel.

1.2 Étape 1 – Installer TrajCenter

Depuis la racine du dépôt, installer l'environnement complet :

```
1 pixi install -e full
```

Listing 1 – Installation rapide de TrajCenter

Vérifier ensuite l'installation et la disponibilité du support ABB :

```
1 pixi run -e full trajcenter version
2 pixi run -e full trajcenter robot check
```

Listing 2 – Vérification de l'installation

Lorsque le client ABB est disponible, la seconde commande affiche :

```
1 Robot ABB API available.
```

Listing 3 – Résultat attendu de la vérification ABB

Cette vérification confirme que les dépendances logicielles sont installées. Elle ne teste pas encore la connexion avec le contrôleur.

1.3 Étape 2 – Créer le fichier de configuration

Créer à la racine du projet un fichier nommé, par exemple, `robot.env`. Adapter son contenu à la cellule utilisée :

```
1 # Connexion ABB Robot Web Services
2 RWS_HOST=192.168.125.1
3 RWS_PORT=80
4 RWS_USER="Default User"
5 RWS_PASSWORD=robotics
6 RWS_TIMEOUT=10.0
7
8 # Localisation du module RAPID TrajCenter
9 TRAJCENTER\_RWS\_TASK=T_ROB1
10 TRAJCENTER\_RWS\_MODULE=TRAJCENTER
11
12 # Store local utilise par le superviseur
13 TRAJCENTER_STORE_ROOT=trajectory_store
14
15 # Comportement du superviseur
```

```
16 TRAJCENTER_MASTERSHIP_RETRIES=3
17 TRAJCENTER_LOG_LEVEL=INFO
```

Listing 4 – Configuration minimale du PC de cellule

Les valeurs doivent être adaptées comme suit :

- `RWS_HOST` contient l'adresse IP ou le nom d'hôte du contrôleur ;
- `RWS_PORT` contient le port HTTP utilisé par RWS ;
- `RWS_USER` et `RWS_PASSWORD` contiennent les identifiants du compte ABB ;
- `TRAJCENTER_RWS_TASK` désigne la tâche dans laquelle le module système est chargé ;
- `TRAJCENTER_RWS_MODULE` contient le nom du module système ;
- `TRAJCENTER_STORE_ROOT` désigne le répertoire contenant les archives disponibles pour le robot.

Le fichier contenant les identifiants ne doit pas être versionné ni transmis avec des mots de passe de production.

Lorsqu'une option est également fournie dans la ligne de commande, elle remplace la valeur correspondante du fichier de configuration. La priorité générale est donc : l'argument de ligne de commande puis la variable d'environnement ou fichier `.env` puis la valeur par défaut.

Les dimensions des tableaux RAPID et les capacités du protocole ne doivent pas être ajoutées au fichier `robot.env`. Elles sont définies dans le module RAPID. La version Python utilise de son côté un profil de protocole compatible pour valider les transferts avant l'écriture.

Dans la version actuelle, le superviseur ne compare pas automatiquement toutes les dimensions du module RAPID avec le profil Python au démarrage. Le module RAPID doit donc être déployé dans une version compatible avec la version de TrajCenter utilisée sur le PC.

1.4 Étape 3 – Préparer la configuration RAPID

Le module `TRAJCENTER.sys` doit être chargé dans la tâche indiquée par `TRAJCENTER_RWS_TASK`. Le nom du module doit correspondre à `TRAJCENTER_RWS_MODULE`.

Au démarrage du programme RAPID, initialiser l'API et la configuration minimale :

```
TRAJCENTER_InitErrors;
TRAJCENTER_InitCellConfig;
```

Listing 5 – Initialisation minimale côté RAPID

La configuration minimale expose `tool0` et `wobj0`. Si la trajectoire utilise d'autres noms, ceux-ci doivent être déclarés avec `TRAJCENTER_UpsertTool` et `TRAJCENTER_UpsertWobj`.

L'exemple suivant définit également des valeurs par défaut utilisables lorsque certaines colonnes sont absentes de la trajectoire :

```
hasDefaultTcpSpeed := TRUE;
defaultTcpSpeed := 100;

hasDefaultZoneType := TRUE;
defaultZoneType := 255;

hasDefaultToolName := TRUE;
defaultToolName := "tool0";

hasDefaultWobjName := TRUE;
```

```
defaultWobjName := "wobj0";

defaultMoveType := moveTypeL;
defaultReadConfs := TRUE;
```

Listing 6 – Exemple de valeurs par défaut côté robot

Dans cet exemple, la vitesse par défaut est de 100 mm/s et la zone 255 correspond à un point fine. Les valeurs réelles doivent être choisies en fonction de l'application et de la configuration de la cellule.

1.5 Étape 4 – Convertir une trajectoire

Placer le fichier source dans un répertoire accessible, par exemple `trajectory_files/`, puis lancer la conversion.

Lorsque le format peut être déterminé à partir de l'extension, l'option `-format` peut être omise :

```
1 pixi run -e full trajcenter convert \
2   trajectory_files/test_basic.xlsx \
3   trajectory_store
```

Listing 7 – Conversion avec détection automatique du format

En cas de succès, TrajCenter affiche le chemin de l'archive créée. Par exemple :

```
1 Created: trajectory_store/test_basic.trajcenter
```

Listing 8 – Exemple de résultat d'une conversion

La conversion est réalisée sans connexion au robot. Elle vérifie la structure du fichier et crée une archive portable, mais elle ne garantit pas encore que la trajectoire est compatible avec une cellule particulière.

1.6 Étape 5 – Vérifier le contenu du store

Afficher les trajectoires disponibles dans le store :

```
1 pixi run -e full trajcenter store list \
2   --store trajectory_store
```

Listing 9 – Vérification du store local

La trajectoire convertie doit apparaître dans la liste avec son nom, son nombre de points, son type de process et son fichier. Ce nom sera ensuite utilisé par le programme RAPID pour sélectionner la trajectoire. L'inspection détaillée d'une archive est présentée dans la section 2.4.

Le nom présent dans les métadonnées sera ensuite utilisé par le programme RAPID pour sélectionner la trajectoire.

1.7 Étape 6 – Démarrer le superviseur

Lancer le superviseur en indiquant le fichier de configuration :

```
1 pixi run -e full trajcenter robot supervise \
2   --env-file robot.env
```

Listing 10 – Démarrage du superviseur ABB RWS

Le store, la tâche RAPID, le module RAPID, le nombre de tentatives de Mastership et le niveau de journalisation sont lus depuis `robot.env`. Les journaux doivent notamment confirmer :

- la connexion au contrôleur ABB ;
- le chemin du store utilisé ;
- la tâche et le module RAPID ciblés ;
- la lecture de la configuration nécessaire sur le contrôleur ;
- la mise en attente des requêtes RAPID.

Le terminal doit rester ouvert pendant l'utilisation. Pour arrêter proprement le superviseur, utiliser `Ctrl+C`.

Les arguments de ligne de commande peuvent remplacer ponctuellement les valeurs du fichier `robot.env`. Leur utilisation est détaillée dans la section 2.5.4.

1.8 Étape 7 – Demander la trajectoire depuis RAPID

Lorsque le superviseur est actif, le programme RAPID peut rafraîchir la liste des trajectoires, demander une trajectoire par son nom et attendre la fin du transfert.

L'exemple suivant charge la trajectoire nommée `test_basic` :

```
TRAJCENTER_InitErrors;
TRAJCENTER_InitCellConfig;

TRAJCENTER_RequestMetaRefresh;
TRAJCENTER_WaitRequestDone 30;

IF transferError = TRUE THEN
    TPWrite "Metadata refresh failed";
    TPWrite "Code:" \Num:=lastErrorCode;
    TPWrite lastError;
    Stop;
ENDIF

TRAJCENTER_RequestTrajByName "test_basic";
TRAJCENTER_WaitTrajectoryReady 120;

TPWrite "Trajectory loaded";
TPWrite "Point count:" \Num:=nbLoadedTrajPoints;
```

Listing 11 – Premier chargement d'une trajectoire depuis RAPID

Le nom transmis à `TRAJCENTER_RequestTrajByName` doit correspondre exactement au nom présent dans les métadonnées de l'archive. Il peut être différent du nom du fichier.

Lorsque le transfert réussit :

- `trajReady` vaut `TRUE` ;
- `nbLoadedTrajPoints` contient le nombre de points valides ;
- les points valides sont disponibles dans `trajData{1..nbLoadedTrajPoints}` ;
- `transferError` vaut `FALSE`.

Si le transfert échoue, consulter :

- les journaux affichés par le superviseur ;
- la variable RAPID `transferError` ;

- le code `lastErrorCode` ;
- le message `lastError`.

1.9 Étape 8 – Vérifier avant l'exécution

Avant tout mouvement, vérifier que :

- la trajectoire chargée est celle attendue ;
- le nombre de points est cohérent ;
- les tools et workobjects correspondent à la cellule ;
- la vitesse et les zones sont adaptées ;
- le premier point peut être rejoint sans collision ;
- les configurations d'axes et les axes externes sont compatibles ;
- les mouvements circulaires sont correctement constitués.

Pour un premier essai, il est recommandé de lire ou d'inspecter les données sans les exécuter :

```
VAR trajCenterPointData firstPoint;

IF nbLoadedTrajPoints > 0 THEN
    TRAJCENTER_GetPoint 1, firstPoint;
    TPWrite "First point loaded";
ENDIF
```

Listing 12 – Inspection du premier point chargé

L'exécution de la trajectoire n'est pas incluse dans cette procédure de démarrage rapide. Elle doit être réalisée uniquement après vérification des positions, des tools, des workobjects, des vitesses, des zones, des configurations d'axes et de l'environnement de la cellule. La procédure d'exécution est décrite dans la section 3.5.2.

1.10 Résumé du premier cycle

Le premier cycle complet peut être résumé ainsi :

1. installer l'environnement Pixi `full` ;
2. configurer la connexion dans `robot.env` ;
3. charger et initialiser le module RAPID `TRAJCENTER.sys` ;
4. convertir un fichier source en archive `.trajcenter` ;
5. vérifier que l'archive apparaît dans le store ;
6. démarrer le superviseur ABB RWS ;
7. demander le rafraîchissement des métadonnées depuis RAPID ;
8. demander la trajectoire par son nom ;
9. contrôler l'état du transfert ;
10. vérifier la trajectoire avant toute exécution.

En cas d'échec, les premières vérifications à effectuer sont :

- l'adresse du contrôleur et les identifiants RWS ;
- la correspondance entre la tâche et le module configurés sur le PC et ceux réellement chargés sur le contrôleur ;

- la présence de l'archive dans le store configuré ;
- la correspondance exacte du nom de la trajectoire ;
- la présence des tools et workobjects demandés ;
- les messages du superviseur et les variables d'erreur RAPID.

Les sections suivantes détaillent chacune de ces opérations ainsi que les méthodes de diagnostic.

2 Installation et utilisation côté PC

Cette section présente l'installation et l'utilisation de TrajCenter sur le poste utilisateur. Deux profils principaux sont distingués :

- le poste de préparation, utilisé pour convertir, exporter et gérer les trajectoires sans connexion à un robot ;
- le PC de cellule, connecté à un contrôleur ABB et utilisé également pour superviser le transfert des trajectoires.

Le projet utilise Pixi pour installer Python et les dépendances adaptées à chaque profil. L'utilisateur n'a pas besoin d'installer Python manuellement, de créer un environnement virtuel ni d'installer séparément les bibliothèques Python.

2.1 Installation

2.1.1 Prérequis

L'installation de TrajCenter nécessite :

- Pixi installé sur le poste ;
- un accès au dépôt TrajCenter ;
- un terminal permettant d'exécuter des commandes ;
- un accès en lecture et en écriture aux répertoires du projet ;
- suffisamment d'espace disque pour les environnements et les trajectoires.

Le fichier `pixi.toml` impose Python 3.12 dans les environnements Pixi. Cette version est téléchargée et gérée automatiquement lors de l'installation.

Les plateformes déclarées dans le workspace sont présentées dans le tableau 2.1.

Plateforme	Système	Architecture
win-64	Windows	Processeur Intel ou AMD 64 bits.
linux-64	Linux	Processeur Intel ou AMD 64 bits.
osx-arm64	macOS	Processeur Apple Silicon.

TABLE 2.1 – Plateformes prises en charge par le workspace Pixi

Les ordinateurs macOS équipés d'un processeur Intel ne font pas partie des plateformes actuellement déclarées.

2.1.2 Organisation des répertoires utilisateur

Trois répertoires sont principalement utilisés pendant l'exploitation de TrajCenter :

Les commandes du CLI permettent d'utiliser d'autres répertoires en indiquant explicitement leur chemin.

Répertoire	Utilisation
trajectory_files/	Fichiers sources à convertir : fichiers Excel, CSV, APT ou modules RAPID.
trajectory_store/	Store local contenant les archives .trajcenter.
trajectory_exports/	Fichiers CSV ou Excel créés par les opérations d'export.

TABLE 2.2 – Principaux répertoires utilisés par TrajCenter

2.1.3 Choix de l'environnement Pixi

TrajCenter fournit plusieurs environnements Pixi. Les fonctions de conversion, d'export et de gestion du store sont communes à tous les profils. L'interface terminal et le client ABB Robot Web Services sont ajoutés en fonction du rôle du poste.

Les deux profils recommandés aux utilisateurs sont :

Poste de préparation : utiliser l'environnement `tui`. Il fournit l'interface terminal et toutes les opérations locales, sans installer les dépendances ABB.

PC connecté au robot : utiliser l'environnement `full`. Il fournit les mêmes opérations locales, l'interface terminal et les fonctions de communication avec le contrôleur ABB.

Les quatre environnements utilisateur sont détaillés dans le tableau 2.3.

Environnement	TUI	ABB RWS	Usage recommandé
default	Non	Non	Traitements locaux en ligne de commande : conversion, export et consultation du store. Convient principalement aux scripts et aux traitements automatisés.
tui	Oui	Non	Profil recommandé pour un poste de préparation non connecté au robot. Il permet d'utiliser le CLI et l'interface terminal.
robot	Non	Oui	Exécution du superviseur ABB en ligne de commande, sans interface Textuel. Ce profil convient à un service automatisé.
full	Oui	Oui	Profil recommandé pour un PC connecté à une cellule ABB. Il fournit toutes les fonctionnalités de TrajCenter.

TABLE 2.3 – Environnements Pixi destinés aux utilisateurs

L'environnement `dev` est réservé au développement, aux tests et aux contrôles de qualité. Il n'est pas destiné aux utilisateurs et n'intervient pas dans les procédures de ce manuel.

2.1.4 Installation d'un poste de préparation

Sur un poste qui sert uniquement à préparer les trajectoires, installer l'environnement `tui` :

```
1 pixi install -e tui
```

Listing 13 – Installation d'un poste de préparation

Cet environnement permet :

- de convertir les fichiers sources ;
- de créer des archives .trajcenter ;

- de consulter le store local ;
- d'exporter les trajectoires en CSV ou en Excel ;
- d'utiliser le CLI ;
- d'utiliser l'interface terminal Textual.

Il n'installe pas le client ABB RWS. Les fonctionnalités nécessitant une connexion au robot ne sont donc pas disponibles.

2.1.5 Installation d'un PC connecté au robot

Sur un PC connecté à un contrôleur ABB, installer l'environnement **full** :

```
1 pixi install -e full
```

Listing 14 – Installation d'un PC connecté au robot

Cet environnement fournit :

- les fonctions de conversion et d'export ;
- la gestion du store local ;
- le CLI TrajCenter ;
- l'interface terminal Textual ;
- le client ABB Robot Web Services ;
- les fonctions de supervision et de transfert vers le robot.

Pour communiquer avec le contrôleur, le poste doit également disposer d'un accès réseau au robot et d'un compte ABB possédant les autorisations nécessaires.

2.1.6 Installations spécialisées

Les environnements **default** et **robot** sont destinés aux utilisations sans interface terminal. Pour installer uniquement les commandes locales :

```
1 pixi install -e default
```

Listing 15 – Installation minimale pour les traitements locaux

Pour exécuter le superviseur ABB sans interface Textual :

```
1 pixi install -e robot
```

Listing 16 – Installation du superviseur ABB sans TUI

Plusieurs environnements peuvent être installés simultanément. Pixi conserve séparément leurs dépendances dans le répertoire **.pixi/**.

2.1.7 Vérification de l'installation

Sur un poste de préparation, vérifier la version installée avec :

```
1 pixi run -e tui trajcenter version
```

Listing 17 – Vérification d'une installation TUI

Sur un PC connecté au robot, vérifier la version et la disponibilité du client ABB avec :

```
1 pixi run -e full trajcenter version
2 pixi run -e full trajcenter robot check
```

Listing 18 – Vérification d'une installation complète

Lorsque les dépendances ABB sont disponibles, la seconde commande affiche :

```
1 Robot ABB API available.
```

Listing 19 – Résultat de la vérification du support ABB

Cette commande vérifie uniquement l’installation du client ABB. Elle ne teste ni le réseau, ni les identifiants, ni la connexion avec le contrôleur.

2.2 Configuration du poste et du robot ciblé

Avant d’utiliser la supervision ABB, il est nécessaire de distinguer les paramètres du poste PC, les règles du format `.trajcenter` et la configuration réelle de la cellule robotisée.

TrajCenter utilise plusieurs sources de configuration, chacune ayant un rôle précis. Cette séparation permet de préparer les trajectoires hors ligne tout en laissant au robot la maîtrise de la configuration de sa cellule.

2.2.1 Qui configure quoi ?

Les responsabilités sont réparties comme indiqué dans le tableau 2.4.

Élément	Rôle	Exemples
Arguments de ligne de commande	Surcharge ponctuelle pour une exécution particulière.	Changer temporairement le contrôleur, la tâche RAPID, le module, le store ou le niveau de journalisation.
Fichier <code>.env</code> ou environnement du système	Configuration habituelle du poste et de la connexion.	Adresse du contrôleur, identifiants RWS, tâche RAPID, module RAPID, store local et comportement du superviseur.
Format <code>.trajcenter</code>	Définit les règles de validité d’une archive locale, indépendamment d’un robot particulier.	Colonnes obligatoires, types de données, métadonnées et cohérence des paramètres process.
Module RAPID TRAJCENTER	Définit les capacités et la configuration réelles de la cellule.	Tools, workobjects, valeurs par défaut, catalogue process et dimensions des tableaux de transfert.

TABLE 2.4 – Répartition des responsabilités de configuration

La règle générale est la suivante :

Les arguments de ligne de commande configurent une exécution particulière, le fichier `.env` configure le poste, le format `.trajcenter` définit une archive valide et le module RAPID définit ce que la cellule robotisée peut réellement accepter.

La conversion ne contacte jamais le contrôleur. Elle peut être réalisée sur un poste de préparation ne disposant ni du client ABB ni d’un accès réseau au robot.

Par conséquent, une archive `.trajcenter` valide n’est pas nécessairement compatible avec toutes les cellules. Une incompatibilité peut notamment provenir :

- d’un nombre de points supérieur à la capacité du module RAPID ;
- d’un tool ou d’un workobject absent de la cellule ;
- d’un type de process non pris en charge ;
- de paramètres process incompatibles ;
- de valeurs obligatoires absentes et sans valeur par défaut côté robot.

Lors de la préparation du transfert, Python refuse les incompatibilités qu'il peut détecter à partir de son profil de protocole et de la configuration lue sur le contrôleur. Les dimensions du module RAPID constituent la limite finale côté robot.

La version du module `TRAJCENTER.sys` déployée sur le contrôleur doit donc rester compatible avec la version du logiciel TrajCenter installée sur le PC. La comparaison automatique de toutes les dimensions Python et RAPID au démarrage n'est pas réalisée dans la version actuelle.

2.2.2 Priorité des paramètres

Lorsqu'un même paramètre est disponible à plusieurs endroits, TrajCenter applique l'ordre de priorité suivant :

argument de ligne de commande	puis	variable d'environnement ou fichier <code>.env</code>
	puis	valeur par défaut

Par exemple, la tâche RAPID utilisée est déterminée dans cet ordre :

1. valeur fournie avec l'option `-task` ;
2. valeur de `TRAJCENTER_RWS_TASK` ;
3. valeur par défaut `T_ROB1`.

Ainsi, la commande suivante utilise temporairement la tâche `T_ROB2`, sans modifier le fichier de configuration :

```
1 pixi run -e full trajcenter robot supervise \
2   --env-file robot.env \
3   --task T_ROB1
```

Listing 20 – Surcharge ponctuelle de la tâche RAPID

La même règle s'applique au module RAPID, au store, au nombre de tentatives de Master-ship et au niveau de journalisation.

2.2.3 Paramètres configurables par l'utilisateur

Les paramètres utilisables par le superviseur sont présentés dans le tableau 2.5.

Les noms des variables doivent être utilisés exactement. En particulier, les variables portant un préfixe `ABB_RWS_` ne remplacent pas les variables `RWS_*` attendues par le client.

2.2.4 Création du fichier de configuration

Pour une utilisation régulière, il est recommandé de regrouper les paramètres dans un fichier local, par exemple `robot.env` :

```
1 # Connexion ABB Robot Web Services
2 RWS_HOST=192.168.125.1
3 RWS_PORT=80
4 RWS_USER="Default User"
5 RWS_PASSWORD=robotics
6 RWS_TIMEOUT=10.0
7
8 # Localisation du module RAPID
9 TRAJCENTER_RWS_TASK=T_ROB1
10 TRAJCENTER_RWS_MODULE=TRAJCENTER
```

Variable	Option CLI	Description
RWS_HOST	-host	Adresse IP ou nom d'hôte du contrôleur ABB.
RWS_PORT	-port	Port HTTP utilisé par ABB Robot Web Services.
RWS_USER	-username	Nom du compte utilisé pour l'authentification RWS.
RWS_PASSWORD	-password	Mot de passe du compte RWS.
RWS_TIMEOUT	-timeout	Timeout des requêtes RWS, en secondes.
TRAJCENTER_RWS_TASK	-task	Tâche RAPID contenant le module TrajCenter. Le repli intégré est T_ROB1.
TRAJCENTER_RWS_MODULE	-module	Nom du module système TrajCenter. Le repli intégré est TRAJCENTER.
TRAJCENTER_STORE_ROOT	-store	Répertoire contenant les archives disponibles pour le superviseur. Le repli intégré est <code>trajectory_store</code> .
TRAJCENTER_MASTERSHIP_RETRIES	-mastership-retries	Nombre de tentatives d'acquisition de la Mastership. La valeur doit être un entier supérieur ou égal à 1.
TRAJCENTER_LOG_LEVEL	-log-level	Niveau de journalisation : <code>DEBUG</code> , <code>INFO</code> , <code>WARNING</code> , <code>ERROR</code> ou <code>CRITICAL</code> .

TABLE 2.5 – Paramètres utilisateur du superviseur

```

11
12 # Store utilise par le superviseur
13 TRAJCENTER_STORE_ROOT=trajectory_store
14
15 # Comportement du superviseur
16 TRAJCENTER_MASTERSHIP_RETRIES=3
17 TRAJCENTER_LOG_LEVEL=INFO

```

Listing 21 – Exemple complet de fichier `robot.env`

Ce fichier doit être adapté à la cellule utilisée. Il ne doit pas être versionné ni diffusé lorsqu'il contient des identifiants réels.

Pour éviter de placer directement le mot de passe dans une commande, il est également possible d'utiliser l'option `-password-env`. Cette option attend le nom d'une variable d'environnement contenant le mot de passe.

2.2.5 Paramètres qui ne doivent pas être placés dans le fichier `.env`

Le fichier `.env` ne doit pas contenir les capacités internes du protocole, notamment :

- le nombre maximal de trajectoires ;
- le nombre maximal de points par trajectoire ;
- le nombre maximal de jeux de paramètres process ;
- le nombre de paramètres par jeu ;
- la taille des tableaux de tools et de workobjects ;
- les codes d'état et d'erreur ;
- les identifiants numériques des types de mouvement ;
- la sentinelle ABB des axes externes inactifs.

Ces éléments ne sont pas des réglages du poste. Les capacités réelles sont définies par le module RAPID chargé sur le contrôleur.

Le fichier interne `trajcenter/robot/constants.py` contient les conventions et le profil de protocole pris en charge par la version installée de TrajCenter. Il n'est pas destiné à être modifié par l'utilisateur et ne remplace pas le fichier `.env`.

Toute modification des dimensions ou des structures RAPID constitue une évolution du protocole. Elle doit être réalisée de manière coordonnée côté robot et côté logiciel TrajCenter.

2.3 Conversion d'une trajectoire

La conversion transforme un fichier source en archive `.trajcenter`. Cette opération est entièrement locale : elle ne se connecte pas au contrôleur et n'utilise ni les tools, ni les workobjects, ni les capacités de la cellule.

Une archive peut donc être préparée sur un poste hors ligne, puis copiée dans le store d'un PC de cellule.

2.3.1 Formats pris en charge

Les formats d'entrée reconnus sont présentés dans le tableau 2.6.

Format	Description
CSV	Fichier tabulaire texte contenant les données de trajectoire.
Excel	Classeur portant notamment l'extension <code>.xlsx</code> , <code>.xlsm</code> ou <code>.xls</code> .
APT	Fichier <code>.apt</code> ou <code>.aptsource</code> issu d'un logiciel de FAO.
RAPID	Module ABB RAPID portant l'extension <code>.mod</code> .

TABLE 2.6 – Formats de trajectoires reconnus en entrée

Lorsque le format n'est pas indiqué explicitement, TrajCenter tente de le déterminer à partir de l'extension du fichier.

2.3.2 Conversion avec le CLI

L'exemple suivant convertit un classeur Excel et crée l'archive dans le store local :

```
1 pixi run -e tui trajcenter convert \
2   trajectory_files/test_basic.xlsx \
3   trajectory_store
```

Listing 22 – Conversion d'un fichier Excel avec le CLI

L'option `-format` permet d'imposer le convertisseur lorsque l'auto-détection n'est pas souhaitée :

```
1 pixi run -e tui trajcenter convert \
2   trajectory_files/test_basic.xlsx \
3   trajectory_store \
4   --format xlsx
```

Listing 23 – Conversion avec sélection explicite du format

L'option `-name` permet de choisir le nom du fichier de sortie sans modifier le nom contenu dans le fichier source :

```
1 pixi run -e tui trajcenter convert \  
2   trajectory_files/test_basic.xlsx \  
3   trajectory_store \  
4   --name production_test
```

Listing 24 – Conversion avec un nom de fichier de sortie imposé

En cas de succès, TrajCenter affiche le chemin de l'archive créée. En cas d'échec, un message commençant par **Error:** indique la cause détectée.

2.3.3 Portabilité et compatibilité

La conversion vérifie la structure et la cohérence locale des données. Elle ne vérifie pas que les noms de tools et de workobjects existent sur un robot particulier.

Il faut donc distinguer :

- une **archive valide**, qui respecte le format **.trajcenter** ;
- une **archive transférable**, qui respecte également les capacités et la configuration du robot ciblé.

La compatibilité fonctionnelle avec la cellule est contrôlée pendant la préparation du transfert, lorsque le superviseur lit les tools, les workobjects, les valeurs par défaut et les types de process exposés par le contrôleur.

Les limites de taille sont contrôlées à partir du profil pris en charge par Python, puis par les dimensions effectivement définies dans le module RAPID. Les versions Python et RAPID doivent donc être déployées ensemble dans des versions compatibles.

2.4 Gestion du store local

Le store est le répertoire contenant les archives **.trajcenter** disponibles. Il sert à la fois à l'utilisateur et au superviseur.

Le store utilisé par le superviseur est sélectionné selon la priorité suivante :

1. option **-store** ;
2. variable **TRAJCENTER_STORE_ROOT** ;
3. répertoire par défaut **trajectory_store**.

Les commandes locales **store list** et **store inspect** utilisent le chemin fourni par leur propre option **-store**. Elles ne nécessitent pas de connexion au robot.

2.4.1 Lister les trajectoires

Pour afficher les archives du store :

```
1 pixi run -e tui trajcenter store list \  
2   --store trajectory_store
```

Listing 25 – Liste des trajectoires du store local

La commande affiche notamment :

- l'index de l'entrée ;
- le nom de la trajectoire ;
- le nombre de points ;
- le type de process ;
- le nom du fichier.

Si aucune archive n'est disponible, le CLI affiche :

```
1 No .trajcenter archives found.
```

Listing 26 – Message affiché lorsque le store est vide

2.4.2 Inspecter une trajectoire

Une trajectoire peut être sélectionnée à partir :

- de son nom dans les métadonnées ;
- du nom complet de son fichier ;
- du nom du fichier sans extension ;
- de son index dans le store.

Par exemple :

```
1 pixi run -e tui trajcenter store inspect test_basic \  
2 --store trajectory_store
```

Listing 27 – Inspection d'une trajectoire du store

L'ordre des entrées du store est important pendant la supervision. L'index `RAPID selectedTrajIndex` correspond à l'ordre des métadonnées transmises par le superviseur. Pour un programme de cellule, une sélection par nom est généralement plus lisible et plus stable qu'une sélection par index.

2.5 Supervision d'un robot ABB

Le superviseur assure la liaison entre le store local et le module `RAPID TRAJCENTER`. Il surveille les requêtes émises côté robot, charge l'archive sélectionnée, vérifie sa compatibilité, résout les valeurs dépendant de la cellule puis écrit les données dans le contrôleur avec ABB Robot Web Services.

2.5.1 Prérequis de communication

La supervision nécessite :

- l'environnement `Pixi full` ou `robot` ;
- un contrôleur ABB utilisant RobotWare 6.x ;
- ABB Robot Web Services activé et accessible ;
- une connexion réseau entre le PC et le contrôleur ;
- un compte ABB disposant des autorisations nécessaires ;
- la possibilité d'acquérir la Mastership `RAPID` ;
- le module `TRAJCENTER.sys` chargé sur le contrôleur ;
- une tâche et un nom de module correspondant à la configuration du poste ;
- un store local accessible.

La commande suivante vérifie uniquement que le support logiciel ABB est installé :

```
1 pixi run -e full trajcenter robot check
```

Listing 28 – Vérification du support logiciel ABB

Elle ne teste ni le réseau, ni les identifiants, ni la présence du module `RAPID` sur le contrôleur.

2.5.2 Rôle du robot pendant le transfert

Pendant la conversion, TrajCenter ne connaît aucune cellule particulière. Pendant le transfert, le superviseur utilise au contraire la configuration réellement exposée par le module RAPID.

Le superviseur lit notamment sur le contrôleur :

- les tools disponibles ;
- les workobjects disponibles ;
- les valeurs par défaut autorisées ;
- le catalogue des types de process ;
- l'état courant du transfert.

Le module RAPID définit également les dimensions réelles des tableaux de transfert. Dans la version actuelle, ces dimensions ne sont pas toutes interrogées automatiquement au démarrage. Python valide les données avec le profil de protocole pris en charge par la version installée, puis le module RAPID impose ses propres bornes au moment de l'écriture.

Lorsqu'une trajectoire utilise un nom de tool ou de workobject, le superviseur recherche ce nom dans la configuration du robot. Si le nom est introuvable et qu'aucune valeur par défaut applicable n'est disponible, le transfert est refusé.

Une trajectoire dépassant une limite prise en charge par Python est refusée avant l'écriture. Une incompatibilité résiduelle avec les dimensions du module RAPID provoque un échec de l'écriture ou du transfert. Le superviseur indique alors l'erreur dans ses journaux et met à jour les variables de diagnostic RAPID lorsque l'opération le permet.

2.5.3 Lancement avec un fichier de configuration

Pour une utilisation normale, lancer le superviseur avec le fichier `robot.env` :

```
1 pixi run -e full trajcenter robot supervise \  
2   --env-file robot.env
```

Listing 29 – Lancement du superviseur ABB

Le superviseur résout alors :

- la connexion RWS ;
- la tâche RAPID ;
- le module RAPID ;
- le chemin du store ;
- le nombre de tentatives de Mastership ;
- le niveau de journalisation.

Au démarrage, les journaux indiquent le store utilisé ainsi que la tâche et le module ciblés. Le terminal doit rester ouvert pendant la supervision.

Pour arrêter proprement le superviseur, utiliser `Ctrl+C`.

2.5.4 Surcharges ponctuelles

Les arguments de ligne de commande permettent de remplacer ponctuellement les valeurs du fichier `robot.env`.

Par exemple, pour utiliser temporairement un autre store :

```
1 pixi run -e full trajcenter robot supervise \  
2   --env-file robot.env \  
3   --store trajectory_store_test
```

Listing 30 – Utilisation ponctuelle d'un autre store

Pour utiliser un autre module et activer les journaux détaillés :

```
1 pixi run -e full trajcenter robot supervise \  
2   --env-file robot.env \  
3   --module TRAJCENTER_TEST \  
4   --log-level DEBUG
```

Listing 31 – Surcharge du module et du niveau de journalisation

Ces options ne modifient pas le fichier `robot.env`. Elles s'appliquent uniquement au processus en cours.

2.5.5 Fonctionnement de la supervision

Lors de son démarrage, le superviseur :

1. charge et valide la configuration du poste ;
2. se connecte au contrôleur ABB ;
3. identifie la tâche et le module RAPID ciblés ;
4. lit la configuration nécessaire au transfert ;
5. vérifie l'accès au store local ;
6. ouvre une souscription RWS sur les requêtes RAPID ;
7. attend une demande de rafraîchissement ou de chargement.

Le catalogue transmis au robot est mis à jour lorsqu'une demande de rafraîchissement des métadonnées est reçue.

Il surveille principalement :

- `refreshMetaRequest`, pour rescanner le store et mettre à jour les métadonnées ;
- `sendTrajRequest`, pour transférer la trajectoire sélectionnée.

Le superviseur ne réalise pas de mouvement robot. Il prépare et transfère les données. L'exécution de la trajectoire reste commandée par le programme RAPID.

2.5.6 Journaux et codes de retour

Pour une utilisation normale, le niveau `INFO` est recommandé. Le niveau `DEBUG` peut être activé temporairement pour afficher davantage d'informations sur les requêtes et les échanges RWS.

Un arrêt normal retourne le code 0. Un code non nul indique une erreur, par exemple :

- une erreur de connexion RWS ;
- un fichier de configuration invalide ;
- un store introuvable ;
- une tâche ou un module RAPID incorrect ;
- une erreur de validation de trajectoire.

Les détails doivent être recherchés dans les dernières lignes du journal et, lorsqu'une requête RAPID était en cours, dans les variables `transferError`, `lastErrorCode` et `lastError`.

2.6 Export d'une trajectoire

L'export transforme une archive `.trajcenter` en un format tabulaire destiné à la consultation, au contrôle ou à la modification externe.

Comme la conversion, l'export est entièrement local et ne nécessite pas de connexion au robot.

Une archive peut être exportée en CSV ou en Excel. L'exemple suivant produit un fichier Excel :

```
1 pixi run -e tui trajcenter export \  
2   trajectory_store/test_basic.trajcenter \  
3   trajectory_exports \  
4   --format excel
```

Listing 32 – Export d'une archive TrajCenter vers Excel

Le format d'export par défaut du CLI est CSV :

```
1 pixi run -e tui trajcenter export \  
2   trajectory_store/test_basic.trajcenter \  
3   trajectory_exports
```

Listing 33 – Export d'une archive au format CSV par défaut

Sur un PC connecté au robot, l'environnement `full` peut être utilisé à la place de l'environnement `tui`.

L'export ne modifie pas l'archive source. Les fichiers produits doivent être reconvertis avant d'être remplacés dans le store si leur contenu est modifié.

2.7 Interface terminal

L'interface utilisateur de TrajCenter est une interface terminal construite avec Textual. Elle donne accès aux principales opérations sans avoir à saisir directement les commandes du CLI.

Elle peut être utilisée dans l'environnement `tui` pour les opérations locales ou dans l'environnement `full` pour disposer également de la supervision ABB.

2.7.1 Lancement de la TUI

Après installation de l'environnement `tui` ou `full`, l'interface peut être lancée avec :

```
1 pixi run trajcenter-tui
```

Listing 34 – Lancement recommandé de l'interface terminal

Cette commande utilise le script `scripts/run_best_tui.py`. Celui-ci recherche un environnement compatible déjà installé. L'environnement `full` est sélectionné en priorité, puis l'environnement `tui`.

Le script de lancement ne crée pas les environnements. Si aucun environnement compatible n'est disponible, installer l'un des profils suivants :

```
1 pixi install -e tui  
2 pixi install -e full
```

Listing 35 – Installation d'un environnement compatible avec la TUI

L'interface peut également être appelée directement par le CLI. Cette méthode permet notamment de sélectionner un autre store au démarrage :

```
1 pixi run -e tui trajcenter tui \  
2   --store trajectory_store
```

Listing 36 – Lancement direct de la TUI avec un store particulier

Dans l'environnement complet, la commande équivalente est :

```
1 pixi run -e full trajcenter tui \
2   --store trajectory_store
```

Listing 37 – Lancement direct de la TUI dans l’environnement complet

2.7.2 Écran d’accueil

Au démarrage, TrajCenter affiche un écran d’accueil contenant les identités Arts et Métiers et LCFC, le titre de l’application et le nom des auteurs.

Depuis cet écran :

- appuyer sur **Entrée** ou sur la barre d’espace pour ouvrir le menu principal ;
- appuyer sur **Q** ou sur **Échap** pour quitter l’application.

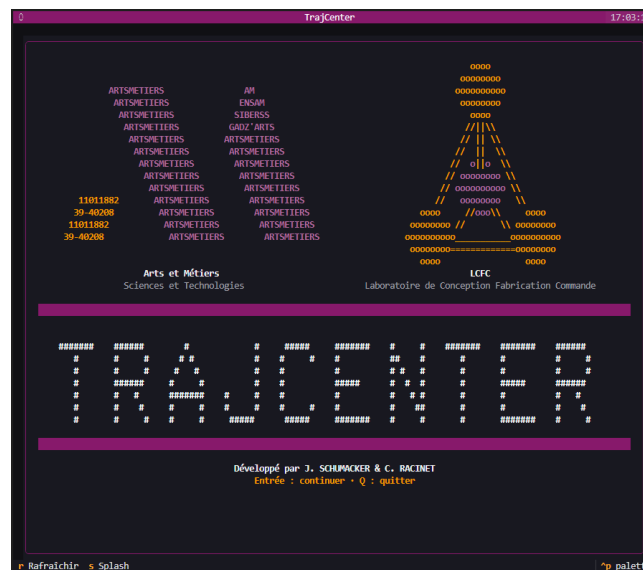


FIGURE 2.1 – Écran d’accueil de TrajCenter

2.7.3 Menu principal

Le menu principal donne accès aux actions suivantes :

Action	Fonction
Convertir une trajectoire	Convertit un fichier CSV, Excel, APT ou RAPID MOD en archive .trajcenter .
Exporter une trajectoire	Exporte une archive .trajcenter en CSV ou en Excel.
Explorer le store local	Affiche les archives présentes dans le store configuré.
Démarrer supervision robot ABB	Ouvre l’écran permettant de démarrer et d’arrêter le superviseur ABB RWS. Cette action nécessite le support robot.
Paramètres	Modifie le store et les paramètres de connexion utilisés pendant la session courante.
Quitter	Ferme l’application.

TABLE 2.7 – Actions disponibles depuis le menu principal

La partie droite de l’écran présente l’état du store local. Elle affiche :

- le chemin du store ;
- le nombre d’archives détectées ;
- jusqu’à cinq entrées du store, avec leur nom, leur nombre de points et leur type de process.

Lorsqu’une action est sélectionnée, sa description apparaît dans la partie inférieure de l’écran.

Si les dépendances ABB ne sont pas installées, l’action de supervision est désactivée et porte la mention « option non installée ». Les opérations de conversion, d’export et de consultation du store restent disponibles.

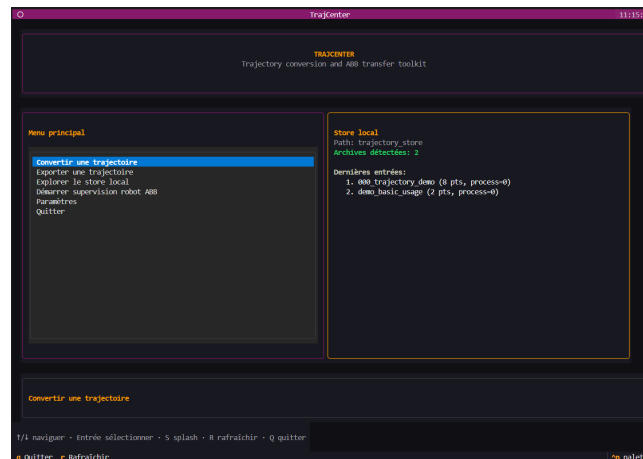


FIGURE 2.2 – Menu principal de TrajCenter

2.7.4 Navigation au clavier

La navigation générale repose sur les touches présentées dans le tableau 2.8.

Touche	Action
Flèches	Déplacent la sélection dans les menus, les listes et les tableaux.
Tab et Maj+Tab	Déplacent le focus entre les champs et les boutons.
Entrée	Sélectionne une action, ouvre une liste ou active le bouton ayant le focus.
B ou Échap	Revient au menu principal depuis les écrans fonctionnels. Sur l’écran initial, Échap ferme l’application.
R	Réinitialise le formulaire de conversion, d’export ou de paramètres. Dans l’écran du store, cette touche recharge la liste des archives.
S	Affiche l’écran d’accueil depuis le menu principal.
X	Demande l’arrêt du superviseur depuis l’écran Robot.
Q	Quitte l’application.
Ctrl+Shift+V	Colle du texte dans un champ, selon le terminal utilisé.

TABLE 2.8 – Principaux raccourcis de l’interface terminal

Les raccourcis réellement actifs sont également rappelés dans le pied de chaque écran.

Champ	Description
Fichier source	Chemin du fichier CSV, Excel, APT, APTSOURCE ou RAPID MOD à convertir. Ce champ est obligatoire.
Dossier destination	Répertoire dans lequel l'archive .trajcenter sera créée. La valeur initiale correspond au store configuré.
Nom de sortie	Nom facultatif de l'archive, sans extension. S'il est vide, le nom du fichier source est utilisé.
Format source	Format imposé par l'utilisateur ou auto-détection à partir de l'extension du fichier.

TABLE 2.9 – Champs de l'écran de conversion

2.7.5 Écran de conversion

L'écran de conversion contient les champs suivants :

Le bouton « Convertir vers **.trajcenter** » lance la conversion. En cas de succès, l'interface affiche le chemin de l'archive créée. En cas d'échec, le message transmis par le convertisseur est affiché dans la zone de statut.

La touche R efface le fichier source et le nom de sortie, restaure le store comme destination et remplace le format sur l'auto-détection.

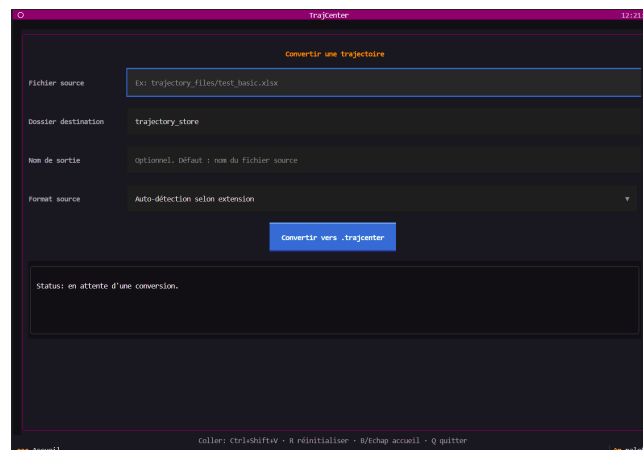


FIGURE 2.3 – Fenêtre de conversion de fichiers

2.7.6 Écran d'export

L'écran d'export contient :

- le chemin de l'archive **.trajcenter** source ;
- le dossier destination, initialisé à **trajectory_exports** ;
- le format de sortie, CSV ou Excel.

Le format sélectionné par défaut est CSV. Le bouton « Exporter la trajectoire » charge l'archive puis crée le fichier dans le dossier indiqué.

Après l'opération, la zone de statut affiche soit le chemin du fichier créé, soit le message d'erreur correspondant.

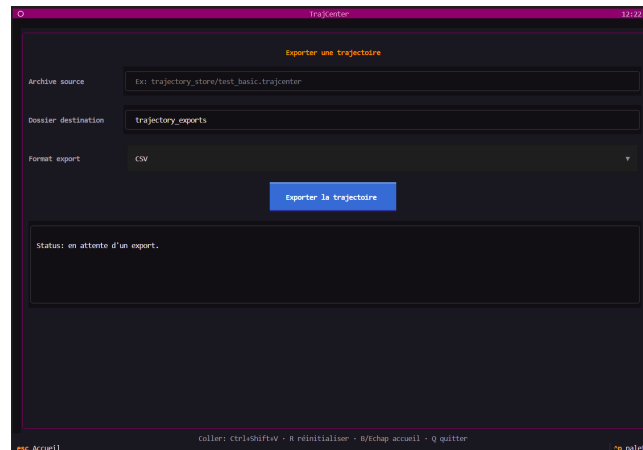


FIGURE 2.4 – Écran d'export d'une archive .trajcenter

2.7.7 Écran du store local

L'écran « Store local TrajCenter » affiche le chemin du store configuré et une table contenant les colonnes suivantes :

- l'index de l'archive ;
- le nom de la trajectoire ;
- le nombre de points ;
- le type de process ;
- le nom du fichier.

La touche R rescane le répertoire et met la table à jour. La zone de statut indique le nombre d'archives détectées ou signale une erreur d'accès au store.

Dans la version actuelle, cet écran permet uniquement de consulter la liste des archives. L'inspection détaillée d'une archive doit être réalisée avec la commande `trajcenter store inspect` décrite dans la section 2.4.

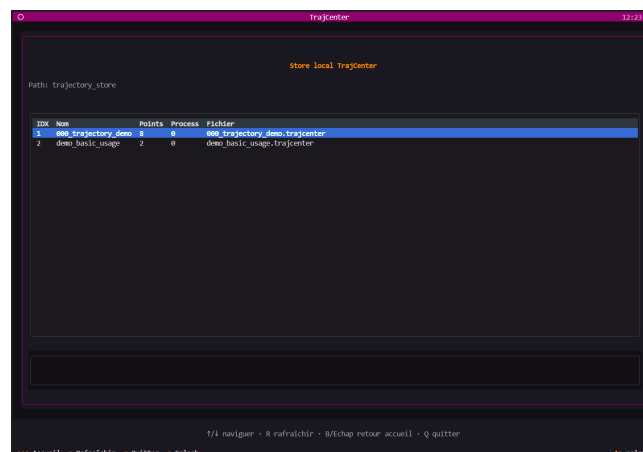


FIGURE 2.5 – Consultation des trajectoires disponibles dans le store local

2.7.8 Écran des paramètres

L'écran des paramètres permet de modifier la configuration partagée par les différents écrans de la TUI.

Paramètre	Description
Store local	Répertoire contenant les archives .trajcenter .
Fichier .env	Fichier facultatif contenant la configuration RWS.
Host robot	Adresse IP ou nom d'hôte du contrôleur ABB.
Port robot	Port HTTP du service RWS.
Utilisateur	Nom du compte ABB utilisé pour l'authentification.
Password env	Nom de la variable d'environnement contenant le mot de passe. Le mot de passe lui-même n'est pas saisi dans cet écran.
Timeout	Timeout des requêtes RWS, en secondes. La virgule et le point sont acceptés comme séparateur décimal.
Task RAPID	Tâche contenant le module TrajCenter. La valeur initiale est T_ROB1 .
Module RAPID	Nom du module système. La valeur initiale est TRAJCENTER .
Mastership retries	Nombre de tentatives d'acquisition de la Mastership. La valeur doit être un entier supérieur ou égal à 1.
Log level	Niveau de journalisation transmis au superviseur. La valeur initiale est INFO .

TABLE 2.10 – Paramètres modifiables depuis la TUI

Les paramètres disponibles sont présentés dans le tableau 2.10.

Le bouton « Appliquer pour cette session » valide les champs et met à jour la configuration en mémoire. Ces modifications ne sont pas enregistrées de manière permanente : elles sont perdues à la fermeture de TrajCenter.

Les valeurs définies dans la TUI sont transmises au superviseur pour la session courante. Elles sont donc prioritaires sur les valeurs correspondantes du fichier **.env**. Avant de démarrer la supervision, l'utilisateur doit vérifier le store, la tâche RAPID, le module RAPID et les paramètres de connexion affichés dans l'interface.

Les valeurs effectivement utilisées sont rappelées dans l'écran de supervision et dans les journaux au démarrage.

La touche **R** ne restaure pas les valeurs d'installation. Elle recharge dans le formulaire la configuration actuellement appliquée.

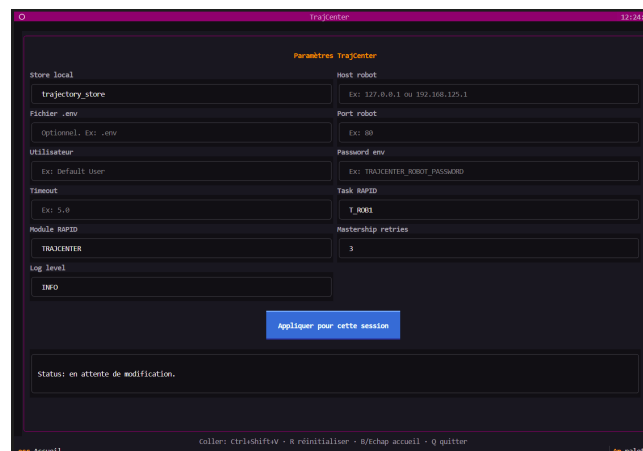


FIGURE 2.6 – Configuration de la session et de la connexion au contrôleur

2.7.9 Écran de supervision robot

L'écran de supervision n'est accessible que lorsque le support ABB est disponible dans l'environnement courant.

Il affiche :

- le chemin du store local ;
- la tâche RAPID utilisée ;
- le nom du module RAPID ;
- l'état du superviseur ;
- les journaux produits par le superviseur.

Le bouton « Démarrer la supervision » lance le superviseur ABB dans un processus séparé. Les sorties standard et les erreurs sont affichées en temps réel dans la zone de journalisation.

Lorsque le superviseur est actif, le bouton devient « Arrêter la supervision ». L'arrêt peut également être demandé avec la touche X.

Il n'est pas possible de revenir au menu principal pendant que le processus est actif. Il faut d'abord demander son arrêt. Si le processus ne répond pas dans le délai prévu, TrajCenter tente de le terminer de force.

Le code de retour du processus est affiché à la fin de la supervision. Un code nul indique une terminaison normale ; un code non nul signale une erreur.

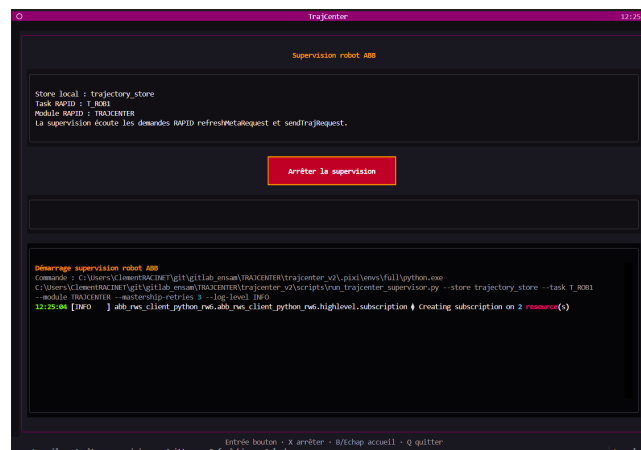


FIGURE 2.7 – Supervision de la connexion au robot et affichage des journaux

2.8 Référence de la ligne de commande

Le CLI permet d'exécuter les fonctions de TrajCenter sans ouvrir l'interface Textual. Il convient notamment aux scripts, aux traitements automatisés et aux postes utilisés sans interaction graphique. Les procédures détaillées de conversion, de gestion du store, d'export et de supervision sont décrites dans les sections précédentes. Cette section fournit uniquement une vue d'ensemble des commandes disponibles.

Pour connaître toutes les options d'une commande, utiliser **-help**. Par exemple :

```
1 pixi run -e full trajcenter robot supervise --help
```

Listing 38 – Affichage des options du superviseur

La syntaxe générale est :

```
1 pixi run -e <environnement> trajcenter <commande> [options]
```

Listing 39 – Syntaxe générale du CLI TrajCenter

L'environnement peut être **default**, **tui**, **robot** ou **full**, selon les fonctionnalités nécessaires.

Pour afficher l'aide générale :

```
1 pixi run -e tui trajcenter --help
```

Listing 40 – Affichage de l'aide générale

Pour afficher l'aide d'une commande particulière :

```
1 pixi run -e tui trajcenter convert --help
```

Listing 41 – Affichage de l'aide d'une commande

Les commandes disponibles sont résumées dans le tableau 2.11.

Commande	Fonction
<code>trajcenter version</code>	Affiche la version installée de TrajCenter.
<code>trajcenter convert</code>	Convertit un fichier source en archive <code>.trajcenter</code> .
<code>trajcenter export</code>	Exporte une archive en CSV ou en Excel.
<code>trajcenter tui</code>	Lance l'interface Textuel avec un store configurable.
<code>trajcenter store list</code>	Liste les archives du store local.
<code>trajcenter store inspect</code>	Affiche les informations détaillées d'une archive sélectionnée par son nom, son fichier, son nom sans extension ou son index.
<code>trajcenter robot check</code>	Vérifie que le support logiciel ABB est disponible. Cette commande ne teste pas la connexion au contrôleur.
<code>trajcenter robot supervise</code>	Lance la boucle de supervision ABB RWS.

TABLE 2.11 – Commandes principales du CLI TrajCenter

Lorsqu'une commande de conversion ou d'export réussit, le CLI affiche le chemin du fichier créé. Lorsqu'une erreur attendue survient, il affiche un message commençant par **Error:** et retourne un code d'échec.

Les sous-commandes **store** et **robot** doivent obligatoirement être complétées par une action. En cas d'oubli, le CLI indique la commande d'aide correspondante.

3 Utilisation côté RAPID

Cette section présente l'utilisation de TrajCenter depuis un programme RAPID. Elle s'adresse aux roboticiens et automaticiens chargés d'installer le module système, de configurer la cellule, de demander une trajectoire et d'exécuter les données transférées.

Le programme RAPID ne lit jamais directement une archive `.trajcenter`. Il émet une requête dans le contrôleur, attend son traitement par le superviseur Python, puis utilise le payload publié dans le module système **TRAJCENTER**.

Les responsabilités sont réparties de la manière suivante :

- le **programme RAPID utilisateur** sélectionne les trajectoires, contrôle les états et déclenche l'exécution ;
- le **superviseur Python** lit le store local, valide l'archive, contrôle sa cohérence avec le catalogue, résout les noms de tools et de workobjects, puis transfère le payload avec ABB Robot Web Services ;
- le **module système TRAJCENTER** fournit les variables d'échange, les routines publiques, les interfaces process et le moteur commun de mouvement.

Dans une utilisation normale, le programme RAPID suit cette séquence :

1. initialiser les erreurs locales TrajCenter ;
2. initialiser et compléter la configuration de cellule ;
3. définir les valeurs par défaut autorisées ;
4. demander le rafraîchissement du catalogue des trajectoires ;
5. demander une trajectoire par son nom ou son index ;
6. attendre la publication complète du payload ;
7. contrôler les données et le type de process chargés ;
8. effectuer les contrôles de sécurité propres à la cellule ;
9. exécuter la trajectoire avec `TRAJCENTER_ExecuteLoaded`.

Toutes les trajectoires utilisent la même API d'exécution. Une trajectoire sans process correspond au cas où `loadedProcessType` vaut `processNone`.

3.1 Installation et principe de fonctionnement

Cette première étape consiste à charger le module système dans la bonne tâche RAPID et à vérifier que le programme utilisateur peut accéder à son API.

3.1.1 Installation du module sur le contrôleur

Le fichier `TRAJCENTER.sys` doit être chargé comme module système dans la tâche RAPID utilisée par TrajCenter, généralement `T_ROB1`.

L'installation peut être réalisée depuis RobotStudio ou depuis l'interface de gestion des modules du contrôleur. Après le chargement, vérifier que :

- le module apparaît sous le nom `TRAJCENTER` ;
- il est chargé comme module système ;
- il se trouve dans la tâche configurée côté PC ;
- aucune erreur de syntaxe ou de chargement RAPID n'est signalée ;
- sa version est compatible avec celle du superviseur Python.

La tâche et le nom du module doivent correspondre aux valeurs configurées sur le PC :

```
1 TRAJCENTER_RWS_TASK=T_ROB1
2 TRAJCENTER_RWS_MODULE=TRAJCENTER
```

Listing 42 – Correspondance entre la configuration PC et le module RAPID

Si le superviseur cible `T_ROB1` et le module `TRAJCENTER`, les symboles utilisés pour les échanges RWS doivent être accessibles dans cette tâche et dans ce module.

Le module peut rester chargé lorsque le superviseur Python est arrêté. En revanche, aucune nouvelle demande de rafraîchissement ou de transfert ne peut aboutir sans superviseur actif.

3.1.2 Principe d'une requête RAPID

Une requête RAPID n'est pas un appel direct à une fonction Python. Le programme modifie une variable persistante du module système. Le superviseur, abonné aux variables concernées par RWS, détecte le changement et effectue le traitement demandé.

Deux requêtes principales sont utilisées :

Le programme utilisateur ne doit normalement pas modifier directement ces variables. Il doit utiliser les routines publiques `TRAJCENTER_*`, qui réalisent les contrôles et les réinitialisations nécessaires.

Requête	Variable RAPID	Résultat attendu
Rafraîchissement du catalogue	<code>refreshMetaRequest</code>	Mise à jour de <code>nbTrajAvailable</code> et du tableau <code>trajectories</code> .
Chargement d'une trajectoire	<code>sendTrajRequest</code>	Validation, résolution et publication du nouveau payload dans <code>trajData</code> , <code>processParams</code> , <code>loadedProcessType</code> et <code>nbLoadedTrajPoints</code> .

TABLE 3.1 – Principales requêtes échangées avec le superviseur

3.1.3 Conditions nécessaires au fonctionnement

Une demande de transfert nécessite simultanément :

- le module `TRAJCENTER.sys` chargé dans la bonne tâche ;
- un superviseur TrajCenter actif sur le PC ;
- une connexion réseau fonctionnelle ;
- ABB Robot Web Services accessible ;
- un compte autorisé à lire et écrire les données RAPID ;
- la possibilité pour le superviseur d'acquérir la Mastership ;
- un store local accessible contenant au moins une archive valide ;
- un catalogue process cohérent avec les capacités réellement implémentées dans le module RAPID.

Si le superviseur est arrêté ou si la connexion est interrompue, la requête RAPID peut rester active jusqu'à l'expiration du délai d'attente.

3.2 Configuration de la cellule

La configuration de la cellule indique au superviseur quels tools et workobjects peuvent être utilisés. Elle définit également les valeurs par défaut autorisées lorsque certaines informations sont absentes d'une trajectoire.

3.2.1 Initialisation de l'API

Avant d'appeler une routine TrajCenter susceptible de lever une erreur RAPID locale, le programme doit initialiser les numéros d'erreur utilisés par l'API.

L'initialisation minimale est la suivante :

```
TRAJCENTER_InitErrors;
TRAJCENTER_InitCellConfig;
```

Listing 43 – Initialisation de l'API TrajCenter

La routine `TRAJCENTER_InitErrors` initialise les erreurs locales TrajCenter. Elle peut être appelée plusieurs fois.

La routine `TRAJCENTER_InitCellConfig` ajoute ou met à jour la configuration minimale contenant :

- le tool `tool0` ;
- le workobject `wobj0`.

Cette routine ne vide pas les tools et workobjects déjà enregistrés. Elle appelle cependant les routines d'ajout ou de mise à jour, ce qui invalide le payload éventuellement chargé.

L'initialisation doit donc généralement être réalisée au démarrage du programme utilisateur, avant le premier chargement d'une trajectoire. Elle ne doit pas être répétée entre le chargement et l'exécution.

3.2.2 Déclaration des tools et des workobjects

Les archives .trajcenter utilisent des noms logiques de tools et de workobjects, par exemple `productionTool` ou `productionWobj`. Leur définition géométrique reste maîtrisée par la cellule robotisée.

Lors du transfert, le superviseur recherche les noms demandés dans les tableaux RAPID `trajTools` et `trajWobjs`. Il associe ensuite les points aux entrées correspondantes en écrivant des index base 1 dans `trajData`.

Un tool et un workobject peuvent être exposés à TrajCenter avec les routines `TRAJCENTER_UpsertTool` et `TRAJCENTER_UpsertWobj` :

```
PERS tooldata productionTool := [
    TRUE,
    [[0, 0, 0], [1, 0, 0, 0]],
    [1, [0, 0, 50], [1, 0, 0, 0],
    0.01, 0.01, 0.01]
];

PERS wobjdata productionWobj := [
    FALSE,
    TRUE,
    "",
    [[0, 0, 0], [1, 0, 0, 0]],
    [[0, 0, 0], [1, 0, 0, 0]]
];

PROC ConfigureTrajCenterCell()

    TRAJCENTER_UpsertTool
        "productionTool",
        productionTool;

    TRAJCENTER_UpsertWobj
        "productionWobj",
        productionWobj;

ENDPROC
```

Listing 44 – Déclaration d'un tool et d'un workobject

Le premier argument est le nom logique utilisé dans les trajectoires. Le second argument est la donnée RAPID réellement utilisée par le robot.

Si le nom existe déjà, la valeur associée est remplacée. Sinon, la routine utilise le premier emplacement libre.

Les noms doivent correspondre exactement à ceux des archives. La comparaison réalisée par le module est une comparaison directe des chaînes.

Toute opération d'ajout ou de mise à jour invalide le payload chargé, y compris lorsque le nom existait déjà. Une trajectoire doit donc être redemandée après une modification de la configuration.

3.2.3 Maintenance des tools et des workobjects

Les principales routines de maintenance sont présentées dans le tableau 3.2.

Routine	Utilisation
TRAJCENTER_UpsertTool	Ajoute un tool ou remplace sa valeur s'il existe déjà.
TRAJCENTER_UpsertWobj	Ajoute un workobject ou remplace sa valeur s'il existe déjà.
TRAJCENTER_DeleteTool	Supprime un tool exposé à TrajCenter.
TRAJCENTER_DeleteWobj	Supprime un workobject exposé à TrajCenter.
TRAJCENTER_ClearTools	Vide le tableau des tools exposés.
TRAJCENTER_ClearWobjs	Vide le tableau des workobjects exposés.
TRAJCENTER_ClearCellConfig	Vide les tableaux de tools et de workobjects.

TABLE 3.2 – Routines de configuration de la cellule

Toutes ces opérations invalident la trajectoire actuellement chargée. Les points transférés contiennent en effet des index associés à la configuration présente au moment du transfert.

Après une modification, appliquer la séquence suivante :

1. mettre à jour le tool ou le workobject ;
2. redemander la trajectoire ;
3. attendre la fin du nouveau transfert ;
4. vérifier que `trajReady` vaut `TRUE` ;
5. vérifier `nbLoadedTrajPoints` ;
6. vérifier `loadedProcessType` ;
7. reprendre l'exécution uniquement après les contrôles de sécurité.

Après un appel à `TRAJCENTER_ClearCellConfig`, il est recommandé d'appeler `TRAJCENTER_InitCellConfig` afin de recréer au minimum `tool0` et `wobj0`.

Les routines de suppression et de vidage sont destinées à la mise en service ou à la maintenance. Elles ne doivent pas être appelées pendant l'exécution d'une trajectoire.

3.2.4 Configuration des valeurs par défaut

Une trajectoire peut omettre certaines données, notamment la vitesse TCP, la zone, le tool ou le workobject. Le superviseur n'utilise une valeur par défaut que lorsque le robot l'autorise explicitement.

Chaque valeur optionnelle est associée à un indicateur `hasDefault*`. L'exemple suivant configure les valeurs par défaut de la cellule :

```
hasDefaultTcpSpeed := TRUE;
defaultTcpSpeed := 100;

hasDefaultZoneType := TRUE;
```

```
defaultZoneType := 255;

hasDefaultToolName := TRUE;
defaultToolName := "productionTool";

hasDefaultWobjName := TRUE;
defaultWobjName := "productionWobj";

defaultMoveType := moveTypeL;
defaultReadConfs := TRUE;
```

Listing 45 – Configuration des valeurs par défaut robot

Dans cet exemple :

- la vitesse TCP par défaut est de 100 mm/s;
- la zone 255 est interprétée comme **fine**;
- le mouvement par défaut est un mouvement linéaire;
- la lecture des configurations d'axes est activée;
- les noms **productionTool** et **productionWobj** doivent exister dans les tableaux de la cellule.

Les valeurs par défaut ne remplacent pas systématiquement les données de la trajectoire. Elles sont utilisées uniquement lorsque la donnée correspondante est absente.

Si une donnée nécessaire est absente et qu'aucune valeur par défaut n'est autorisée, le superviseur refuse le transfert et renseigne les variables de diagnostic.

3.2.5 Routine de configuration recommandée

Il est recommandé de centraliser la configuration TrajCenter dans une routine dédiée du programme de cellule :

```
PROC ConfigureTrajCenter()

    TRAJCENTER_InitErrors;
    TRAJCENTER_InitCellConfig;

    TRAJCENTER_UpsertTool
        "productionTool",
        productionTool;

    TRAJCENTER_UpsertWobj
        "productionWobj",
        productionWobj;

    hasDefaultTcpSpeed := TRUE;
    defaultTcpSpeed := 100;

    hasDefaultZoneType := TRUE;
    defaultZoneType := 255;

    hasDefaultToolName := TRUE;
    defaultToolName := "productionTool";
```

```
hasDefaultWobjName := TRUE;
defaultWobjName := "productionWobj";

defaultMoveType := moveTypeL;
defaultReadConfs := TRUE;
```

ENDPROC

Listing 46 – Organisation recommandée de la configuration

Cette organisation sépare la configuration de la cellule du cycle de production. Elle facilite également la vérification des noms et des valeurs utilisés pendant la mise en service.

Cette routine doit être exécutée avant le chargement de la trajectoire. Les appels à `TRAJCENTER_InitCellConfig`, `TRAJCENTER_UpsertTool` et `TRAJCENTER_UpsertWobj` invalident le payload courant.

3.3 Chargement d'une trajectoire

Le chargement comporte deux opérations distinctes :

1. rafraîchir le catalogue des trajectoires disponibles ;
2. demander le transfert de la trajectoire sélectionnée.

3.3.1 Rafraîchissement des métadonnées

Avant de sélectionner une trajectoire, le robot doit connaître le contenu du store local côté PC.

La routine `TRAJCENTER_RequestMetaRefresh` active la demande de rafraîchissement. Le superviseur inspecte alors le store local et met à jour :

- `nbTrajAvailable`, qui contient le nombre de trajectoires disponibles ;
- `trajectories`, qui contient leurs métadonnées.

Le programme doit attendre la fin de l'opération avec `TRAJCENTER_WaitRequestDone` :

```
TRAJCENTER_RequestMetaRefresh;
TRAJCENTER_WaitRequestDone 30;

IF transferError = TRUE THEN
    TPWrite "Metadata refresh failed";
    TPWrite "Code:" \Num:=lastErrorCode;
    TPWrite lastError;
    Stop;
ENDIF
```

Listing 47 – Rafraîchissement du catalogue des trajectoires

L'argument 30 correspond ici au délai maximal d'attente, en secondes.

Après un rafraîchissement réussi, seules les entrées suivantes sont valides :

```
trajectories{1}
...
```

```
trajectories{nbTrajAvailable}
```

Listing 48 – Bornes valides du catalogue

Les entrées situées au-delà de `nbTrajAvailable` doivent être ignorées. Elles peuvent encore contenir des données provenant d'un rafraîchissement précédent.

La routine `TRAJCENTER_WaitRequestDone` retourne normalement lorsque `transferError` devient `TRUE`. Le programme appelant doit donc toujours contrôler ce flag après l'attente.

Le rafraîchissement des métadonnées n'invalidé pas le payload déjà chargé. Une trajectoire précédemment publiée peut donc rester valide pendant un simple rafraîchissement du catalogue.

3.3.2 Sélection par nom

La sélection par nom est recommandée dans les programmes de production. Elle est généralement plus lisible et plus stable qu'une sélection par index.

L'exemple suivant demande une trajectoire nommée `000_trajectory_demo` :

```
TRAJCENTER_RequestTrajByName "000_trajectory_demo";  
TRAJCENTER_WaitTrajectoryReady 120;
```

Listing 49 – Chargement d'une trajectoire par nom

Le nom demandé doit correspondre exactement au nom présent dans les métadonnées. Il peut être différent du nom du fichier `.trajcenter`.

La routine `TRAJCENTER_RequestTrajByName` :

1. recherche le nom avec `TRAJCENTER_FindTrajectoryIndex`;
2. lève `ERR_TRAJCENTER_TRAJ_NOT_FOUND` si le nom est absent ;
3. appelle `TRAJCENTER_RequestTrajectory` avec l'index trouvé.

3.3.3 Sélection par index

Une trajectoire peut également être demandée à partir de son index :

```
TRAJCENTER_RequestTrajectory 1;  
TRAJCENTER_WaitTrajectoryReady 120;
```

Listing 50 – Chargement d'une trajectoire par index

L'index doit être compris entre 1 et `nbTrajAvailable`.

L'ordre du catalogue peut évoluer lorsque le contenu du store change. La sélection par index convient donc surtout à une interface opérateur affichant le catalogue courant ou à une utilisation contrôlée après rafraîchissement.

3.3.4 Invalidation du payload précédent

La routine `TRAJCENTER_RequestTrajectory` invalide immédiatement le payload précédent avant d'activer la nouvelle requête.

L'état d'invalidation est le suivant :

```
trajReady := FALSE;
nbLoadedTrajPoints := 0;
loadedProcessType := processNone;
```

Listing 51 – Invalidation précédant une nouvelle demande

Les anciennes valeurs de `trajData` et de `processParams` ne sont pas nécessairement effacées. Elles ne sont cependant plus exploitables, car le payload n'est plus déclaré valide.

Le programme utilisateur ne doit donc jamais utiliser les valeurs résiduelles des tableaux lorsque `trajReady` vaut `FALSE`.

3.3.5 Publication du nouveau payload

Après réception de la requête, le superviseur charge l'archive sélectionnée, vérifie sa cohérence avec les métadonnées publiées, lit le contexte robot puis résout les données dépendantes de la cellule.

Le transfert est ensuite réalisé en trois phases :

1. **invalidation** du payload précédent ;
2. **écriture** de `processParams`, `trajData`, `loadedProcessType` et `nbLoadedTrajPoints` ;
3. **publication** avec `trajReady` à `TRUE`, puis remise de `sendTrajRequest` à `FALSE`.

Le type de process n'est donc pas choisi manuellement par le programme utilisateur. Il est fourni par l'archive chargée, validé par Python puis écrit dans `loadedProcessType`.

3.3.6 Attente de la fin du transfert

La demande de trajectoire est asynchrone. Le programme doit attendre avec :

```
TRAJCENTER_WaitTrajectoryReady 120;
```

Listing 52 – Attente de la fin d'un transfert

Lorsque cette routine se termine normalement :

- `sendTrajRequest` vaut `FALSE` ;
- `trajReady` vaut `TRUE` ;
- `transferError` vaut `FALSE` ;
- `nbLoadedTrajPoints` est strictement positif ;
- `loadedProcessType` contient le process du payload chargé ;
- les points peuvent être inspectés ou exécutés.

Si `transferError` devient `TRUE`, la routine lève `ERR_TRAJCENTER_NOT_READY`. Si le délai expire, elle lève `ERR_TRAJCENTER_TIMEOUT`.

Le timeout doit être adapté à la taille maximale des trajectoires, aux performances du contrôleur et aux conditions réseau.

3.3.7 Contrôle du type de process chargé

Chaque entrée du catalogue contient un champ `trajectories{i}.processType`. Le payload publié contient de son côté `loadedProcessType`.

Après un chargement, le programme peut vérifier leur cohérence :

```

IF selectedTrajIndex < 1 OR
  selectedTrajIndex > nbTrajAvailable THEN

  TPWrite "Invalid selected trajectory";
  Stop;

ENDIF

IF loadedProcessType <>
  trajectories[selectedTrajIndex].processType THEN

  TPWrite "Loaded process mismatch";
  Stop;

ENDIF

```

Listing 53 – Vérification du type de process chargé

Cette vérification est complémentaire aux contrôles réalisés par le superviseur.

Dans la version actuelle du module, seul le process **NONE** est publié dans le catalogue **processTypes**. Les identifiants **ACF**, **AAK** et **PUSHCORP** sont réservés, mais leurs implémentations ne sont pas encore déclarées opérationnelles.

3.3.8 États et diagnostic du transfert

Les principales variables de diagnostic sont :

Variable	Signification
trajReady	Indique que le payload complet a été publié et peut être utilisé.
transferError	Indique qu'une erreur est survenue pendant le traitement de la dernière requête.
lastErrorCode	Contient le code numérique du dernier état ou de la dernière erreur écrit par Python.
lastError	Contient un message court destiné au diagnostic.
transferProgress	Indique la progression du traitement, lorsqu'elle est disponible.
nbLoadedTrajPoints	Contient le nombre de points valides du payload chargé.
loadedProcessType	Contient le type de process associé au payload chargé. Cette valeur n'est exploitable que si trajReady vaut TRUE .

TABLE 3.3 – Principales variables d'état RAPID

Les routines d'attente ne réagissent pas de la même manière aux erreurs :

Il est donc inutile de placer systématiquement un test de **transferError** immédiatement après **TRAJCENTER_WaitTrajectoryReady**. Si une erreur de transfert est détectée, l'exécution est redirigée vers le gestionnaire **ERROR** avant d'atteindre ce test.

Les erreurs locales RAPID peuvent notamment signaler :

- un nom ou un index de trajectoire invalide ;
- un index de point hors limites ;

Routine	Comportement en cas d'erreur
TRAJCENTER_WaitRequestDone	Retourne normalement lorsque <code>transferError</code> devient <code>TRUE</code> . Le programme doit contrôler explicitement ce flag.
TRAJCENTER_WaitTrajectoryReady	Lève <code>ERR_TRAJCENTER_NOT_READY</code> si une erreur de transfert est signalée ou si aucun point valide n'a été publié.

TABLE 3.4 – Comportement des routines d'attente

- un tool ou un workobject invalide ;
- une trajectoire qui n'est pas prête ;
- un type de process invalide ;
- un process qui n'est pas implémenté ;
- un index de paramètres process invalide ;
- l'expiration d'un timeout.

3.4 Inspection des données chargées

Avant toute exécution, il est recommandé de vérifier que la trajectoire chargée correspond bien à la demande, que son nombre de points est cohérent et que son type de process est attendu.

3.4.1 Contenu d'un point

Chaque point transféré contient :

- le type de mouvement ;
- le `robtarget` ;
- la vitesse TCP ;
- la zone ;
- la politique de lecture des configurations d'axes ;
- l'index du tool ;
- l'index du workobject ;
- l'index éventuel d'un jeu de paramètres process.

Les tools et workobjects ne sont pas dupliqués dans chaque point. Le point contient des index base 1 associés aux tableaux configurés dans le module système.

3.4.2 Bornes valides

Le programme doit toujours respecter les bornes indiquées par les compteurs et les indicateurs RAPID :

- seules les entrées `trajectories{1..nbTrajAvailable}` contiennent des métadonnées valides ;
- les données du payload ne sont exploitables que si `trajReady` vaut `TRUE` ;
- seules les entrées `trajData{1..nbLoadedTrajPoints}` contiennent les points valides ;
- `loadedProcessType` n'est valide que pour le payload actuellement publié ;
- seuls les jeux de paramètres process référencés par les points valides doivent être utilisés.

Les entrées situées au-delà de ces bornes peuvent contenir des données obsolètes provenant d'un transfert précédent. Leur présence ne signifie pas qu'elles appartiennent au payload courant.

3.4.3 Routines d'accès

Le nombre de points peut être obtenu avec `TRAJCENTER_GetLoadedPointCount`. Un point peut ensuite être copié dans une variable utilisateur avec `TRAJCENTER_GetPoint` :

```
VAR num pointCount;
VAR trajCenterPointData pointData;

IF trajReady = FALSE THEN
    TPWrite "No valid trajectory";
    Stop;
ENDIF

pointCount := TRAJCENTER_GetLoadedPointCount;

IF pointCount > 0 THEN
    TRAJCENTER_GetPoint 1, pointData;
    TPWrite "First point available";
ENDIF
```

Listing 54 – Lecture contrôlée d'un point chargé

La routine `TRAJCENTER_GetPoint` contrôle l'index demandé par rapport à `nbLoadedTrajPoints`.

Les routines `TRAJCENTER_GetTool` et `TRAJCENTER_GetWobj` permettent d'accéder aux configurations associées :

```
VAR tooldata pointTool;
VAR wobjdata pointWobj;

TRAJCENTER_GetTool pointData.toolIndex, pointTool;
TRAJCENTER_GetWobj pointData.wobjIndex, pointWobj;
```

Listing 55 – Lecture du tool et du workobject associés à un point

3.4.4 Accès aux paramètres process

Chaque point peut référencer un jeu de paramètres process avec son champ `processParamIndex`.

La convention est la suivante :

- la valeur 0 signifie qu'aucun jeu de paramètres n'est associé au point ;
- une valeur positive désigne une ligne de `processParams` ;
- un nom vide dans un slot de paramètre indique que ce slot est inutilisé ;
- les valeurs process transportées par le protocole sont numériques.

La présence d'un jeu de paramètres peut être contrôlée avec `TRAJCENTER_PointHasProcessParams`. Un paramètre peut ensuite être recherché par son nom :

```
VAR bool found;
VAR num processValue;
```

```
IF TRAJCENTER_PointHasProcessParams(1) THEN

    TRAJCENTER_GetProcessParam
        1,
        "example_parameter",
        found,
        processValue;

    IF found THEN
        TPWrite "Process parameter found";
    ENDIF

ENDIF
```

Listing 56 – Lecture contrôlée d'un paramètre process

Ces routines doivent être préférées à une lecture directe du tableau `processParams`, car elles centralisent les contrôles d'index.

3.5 Exécution d'une trajectoire

Le module fournit une API d'exécution unique pour les trajectoires avec ou sans process métier :

`TRAJCENTER_ExecuteLoaded`

L'exécution reste néanmoins sous la responsabilité du programme de cellule.

3.5.1 Précautions avant l'exécution

Un transfert réussi garantit uniquement que les données ont été acceptées par le protocole TrajCenter. Il ne garantit pas :

- que toutes les positions sont atteignables ;
- que la trajectoire est exempte de collision ;
- que le premier point peut être rejoint en sécurité ;
- que les configurations d'axes sont adaptées ;
- que les axes externes sont dans la position attendue ;
- que les vitesses et les zones conviennent au procédé ;
- que le process est configuré correctement ;
- que la trajectoire est sûre dans l'état courant de la cellule.

Avant toute exécution, vérifier :

- le nom de la trajectoire chargée ;
- le nombre de points ;
- la valeur de `loadedProcessType` ;
- le tool et le workobject associés ;
- les données mécaniques et la charge du tool ;
- les vitesses et les zones ;
- les configurations d'axes ;
- la cohérence des mouvements circulaires ;

- l'état des axes externes ;
- l'état sûr des équipements process ;
- le dégagement de la zone de travail.

Les premiers essais doivent être réalisés dans RobotStudio ou en mode manuel à vitesse réduite, conformément aux règles de sécurité applicables à la cellule.

3.5.2 Exécution avec l'API unifiée

La routine `TRAJCENTER_ExecuteLoaded` constitue l'unique point d'entrée recommandé pour exécuter une trajectoire chargée. Elle utilise les instructions `MoveL`, `MoveJ` ou `MoveC` selon le type de mouvement associé à chaque point.

Le label de cette sous-section conserve son ancien nom afin de préserver les références présentes dans le reste du manuel. L'API décrite ici est toutefois l'API unifiée actuelle.

Son utilisation est la suivante :

```
TRAJCENTER_ExecuteLoaded 500, 5000, 1000;
```

Listing 57 – Exécution d'une trajectoire chargée

La vitesse TCP provient du point transféré ou de la valeur par défaut appliquée pendant la résolution. Les trois arguments complètent les autres composantes du `speeddata` ABB.

Argument	Type	Utilisation
<code>oriSpeed</code>	num	Vitesse d'orientation.
<code>leaxSpeed</code>	num	Vitesse des axes externes linéaires.
<code>reaxSpeed</code>	num	Vitesse des axes externes rotatifs.

TABLE 3.5 – Arguments de `TRAJCENTER_ExecuteLoaded`

La routine vérifie d'abord :

- que `trajReady` vaut `TRUE` ;
- que `nbLoadedTrajPoints` est compris entre 1 et `maxTrajPointCount`.

Elle copie ensuite `loadedProcessType` dans une variable locale nommée `activeProcessType`. Cette copie garantit que le même type de process est utilisé pendant tout le cycle d'exécution.

Le cycle d'exécution est le suivant :

1. appel de `TRAJCENTER_ProcessBegin` ;
2. parcours du payload avec `TRAJCENTER_ExecuteOneMotion` ;
3. appels process avant et après chaque instruction de mouvement ;
4. appel de `TRAJCENTER_ProcessEnd` après le dernier mouvement ;
5. restauration des politiques de configuration de mouvement.

Pour chaque mouvement, le moteur commun :

1. lit le point courant dans `trajData` ;
2. construit le `speeddata` ;
3. convertit la zone numérique en `zonedata` ;
4. charge le `tooldata` dans `trajExecTool` ;
5. charge le `wobjdata` dans `trajExecWobj` ;

6. applique la politique `readConfs` ;
7. appelle `TRAJCENTER_ProcessBeforeMotion` ;
8. exécute l'instruction de mouvement ;
9. appelle `TRAJCENTER_ProcessAfterMotion` ;
10. avance l'index de parcours.

Lorsque `loadedProcessType` vaut `processNone`, les hooks `process` retournent immédiatement sans réaliser d'action. La trajectoire utilise néanmoins le même moteur de mouvement que les trajectoires avec `process`.

Dans la version actuelle du module, seul le `process NONE` est publié dans le catalogue `processTypes`. Les `process ACF`, `AAK` et `PUSHCORP` ne doivent pas être ajoutés au catalogue avant que leurs routines soient complètement implémentées.

En cas d'erreur pendant l'exécution, la routine :

1. appelle `TRAJCENTER_ProcessAbort` avec le type de `process` actif ;
2. restaure les politiques de configuration de mouvement ;
3. propage l'erreur originale au programme appelant avec `RAISE`.

Cette routine ne doit être appelée que lorsque `TRAJCENTER_WaitTrajectoryReady` s'est terminée normalement et après validation de la trajectoire dans les conditions réelles de la cellule.

3.5.3 Mouvements circulaires

Un mouvement `MoveC` nécessite un point de passage et un point d'arrivée. Dans la représentation `TrajCenter`, ces deux cibles sont portées par deux entrées consécutives de type `moveTypeC`.

La routine `TRAJCENTER_ExecuteOneMotion` vérifie que le second point existe et qu'il est également de type `moveTypeC`.

Elle vérifie aussi que les deux points utilisent les mêmes propriétés d'exécution :

- la même vitesse `TCP` ;
- le même `tool` ;
- le même `workobject` ;
- la même zone ;
- la même politique `readConfs` ;
- le même index de paramètres `process`.

Si l'une de ces conditions n'est pas respectée, la routine lève l'erreur locale `ERR_TRAJCENTER_BAD_MOVEC`.

Une trajectoire ne doit donc pas :

- contenir un point circulaire isolé ;
- se terminer par le premier point d'une paire circulaire ;
- interrompre une paire circulaire par un autre type de mouvement ;
- utiliser des propriétés d'exécution différentes entre les deux points de la paire.

Pour un mouvement circulaire, les hooks `TRAJCENTER_ProcessBeforeMotion` et `TRAJCENTER_ProcessAfterMotion` sont appelés une seule fois. L'index transmis correspond au point de passage. Le point d'arrivée se trouve à l'index suivant.

Hook	Rôle
TRAJCENTER_ProcessBegin	Initialise le process avant le premier mouvement.
TRAJCENTER_ProcessBeforeMotion	Applique les paramètres process avant le mouvement courant.
TRAJCENTER_ProcessAfterMotion	Réalise les actions nécessaires après l'instruction de mouvement.
TRAJCENTER_ProcessEnd	Termine normalement le process après le dernier mouvement.
TRAJCENTER_ProcessAbort	Tente de remettre le process dans un état sûr après une erreur.

TABLE 3.6 – Hooks process utilisés pendant l'exécution

3.5.4 Comportement des hooks process

L'exécution générique utilise cinq points d'extension process :

Pour le process NONE, les hooks ne réalisent aucune action.

Une action exécutée dans le hook **AfterMotion** n'est pas nécessairement synchronisée avec le passage physique exact au point lorsqu'une zone non fine est utilisée. Un procédé exigeant une synchronisation précise doit employer une stratégie ABB adaptée, par exemple des triggers, des instructions process spécialisées ou un exécuteur dédié.

3.6 Exemple complet d'intégration

L'exemple suivant regroupe la configuration, le rafraîchissement du catalogue, le chargement et l'exécution avec l'API unifiée. Il doit être adapté aux données, aux équipements et aux règles de sécurité de la cellule.

MODULE TRAJCENTER_PRODUCTION_EXAMPLE

```
CONST string productionTrajectory :=
    "000_trajectory_demo";
```

```
CONST num refreshTimeout := 30;
CONST num transferTimeout := 120;
```

```
CONST num oriSpeed := 500;
CONST num leaxSpeed := 5000;
CONST num reaxSpeed := 1000;
```

```
PERS tooldata productionTool := [
    TRUE,
    [[0, 0, 0], [1, 0, 0, 0]],
    [1, [0, 0, 50], [1, 0, 0, 0],
    0.01, 0.01, 0.01]
];
```

```
PERS wobjdata productionWobj := [
    FALSE,
    TRUE,
```

```
    "" ,  
    [[0, 0, 0], [1, 0, 0, 0]],  
    [[0, 0, 0], [1, 0, 0, 0]]  
];
```

```
PROC ConfigureTrajCenter()
```

```
    TRAJCENTER_InitErrors;  
    TRAJCENTER_InitCellConfig;  
  
    TRAJCENTER_UpsertTool  
        "productionTool",  
        productionTool;  
  
    TRAJCENTER_UpsertWobj  
        "productionWobj",  
        productionWobj;  
  
    hasDefaultTcpSpeed := TRUE;  
    defaultTcpSpeed := 100;  
  
    hasDefaultZoneType := TRUE;  
    defaultZoneType := 255;  
  
    hasDefaultToolName := TRUE;  
    defaultToolName := "productionTool";  
  
    hasDefaultWobjName := TRUE;  
    defaultWobjName := "productionWobj";  
  
    defaultMoveType := moveTypeL;  
    defaultReadConfs := TRUE;
```

```
ENDPROC
```

```
PROC RefreshMetadata()
```

```
    TRAJCENTER_RequestMetaRefresh;  
    TRAJCENTER_WaitRequestDone refreshTimeout;  
  
    IF transferError = TRUE THEN  
        TPWrite "Metadata refresh failed";  
        TPWrite "Code:" \Num:=lastErrorCode;  
        TPWrite lastError;  
        Stop;  
    ENDIF  
  
    IF nbTrajAvailable < 1 THEN  
        TPWrite "No trajectory available";
```

```
        Stop;
    ENDIF

ENDPROC

PROC LoadTrajectory(string trajectoryName)

    TRAJCENTER_RequestTrajByName trajectoryName;
    TRAJCENTER_WaitTrajectoryReady transferTimeout;

    TPWrite "Trajectory loaded";
    TPWrite "Point count:" \Num:=nbLoadedTrajPoints;
    TPWrite "Process type:" \Num:=loadedProcessType;

ENDPROC

PROC ExecuteTrajectory()

    IF trajReady = FALSE THEN
        RAISE ERR_TJC_NOT_READY;
    ENDIF

    IF nbLoadedTrajPoints < 1 THEN
        RAISE ERR_TJC_NOT_READY;
    ENDIF

    ! Perform cell-specific safety checks here.

    TRAJCENTER_ExecuteLoaded
        oriSpeed,
        leaxSpeed,
        reaxSpeed;

ENDPROC

PROC main()

    ConfigureTrajCenter;
    RefreshMetadata;
    LoadTrajectory productionTrajectory;
    ExecuteTrajectory;

ERROR

    IF ERRNO = ERR_TJC_NOT_READY THEN

        TPWrite "Trajectory not ready";
        TPWrite "Code:" \Num:=lastErrorCode;
```

```

        TPWrite lastError;

    ELSEIF ERRNO = ERR_TJC_TIMEOUT THEN

        TPWrite "TrajCenter timeout";

    ELSEIF ERRNO =
        ERR_TJC_PROCESS_NOT_IMPLEMENTED THEN

        TPWrite "Process not implemented";
        TPWrite "Process:" \Num:=loadedProcessType;

    ELSEIF ERRNO =
        ERR_TJC_BAD_PROCESS_TYPE THEN

        TPWrite "Invalid process type";
        TPWrite "Process:" \Num:=loadedProcessType;

    ELSEIF ERRNO =
        ERR_TJC_BAD_MOVEC_PAIR THEN

        TPWrite "Invalid MoveC pair";

    ELSE

        TPWrite "Unexpected RAPID error";
        TPWrite "ERRNO:" \Num:=ERRNO;
        TPWrite "Code:" \Num:=lastErrorCode;
        TPWrite lastError;

    ENDIF

    Stop;

ENDPROC

ENDMODULE

```

Listing 58 – Exemple structuré d'intégration de TrajCenter

Cet exemple sépare les responsabilités suivantes :

- **ConfigureTrajCenter** configure l'interface avec la cellule ;
- **RefreshMetadata** met à jour le catalogue ;
- **LoadTrajectory** demande le transfert et affiche les informations du payload chargé ;
- **ExecuteTrajectory** réalise les contrôles locaux puis appelle l'API unifiée ;
- **main** organise le cycle de haut niveau.

La configuration des tools et workobjects doit être effectuée avant le chargement. Les routines **TRAJCENTER_InitCellConfig**, **TRAJCENTER_UpsertTool** et **TRAJCENTER_UpsertWobj** invalident le payload actuellement chargé.

Le rafraîchissement du catalogue peut être limité au démarrage ou aux moments où le contenu du store est susceptible d'avoir changé. Il n'est pas nécessaire de rafraîchir les méta-

données avant chaque transfert si le contenu du store est resté identique.

3.7 Procédure d'utilisation courante

Après la mise en service initiale, l'utilisation courante peut être résumée de la manière suivante :

1. démarrer le superviseur sur le PC ;
2. initialiser l'API TrajCenter dans le programme RAPID ;
3. vérifier la configuration des tools et workobjects ;
4. configurer les valeurs par défaut autorisées ;
5. rafraîchir le catalogue si le contenu du store a changé ;
6. demander la trajectoire par son nom ;
7. attendre la publication complète du payload ;
8. vérifier `trajReady` ;
9. vérifier `nbLoadedTrajPoints` ;
10. vérifier `loadedProcessType` ;
11. effectuer les contrôles propres à la cellule ;
12. exécuter la trajectoire avec `TRAJCENTER_ExecuteLoaded`.

Le cycle RAPID minimal, après initialisation de la cellule et rafraîchissement du catalogue, est donc :

```
TRAJCENTER_RequestTrajByName "000_trajectory_demo";
TRAJCENTER_WaitTrajectoryReady 120;

TPWrite "Point count:" \Num:=nbLoadedTrajPoints;
TPWrite "Process type:" \Num:=loadedProcessType;

! Perform cell-specific safety checks here.

TRAJCENTER_ExecuteLoaded 500, 5000, 1000;
```

Listing 59 – Cycle minimal de chargement et d'exécution

En cas d'échec, relever simultanément :

- le message affiché sur le pupitre ou dans le journal RAPID ;
- la valeur de `ERRNO` lorsqu'une erreur locale a été levée ;
- la valeur de `lastErrorCode` ;
- le contenu de `lastError` ;
- la valeur de `transferProgress` ;
- la valeur de `loadedProcessType`, si le payload avait été publié ;
- les dernières lignes du journal du superviseur Python.

Ces informations permettent de distinguer :

- une erreur locale RAPID ;
- une erreur de validation de trajectoire ;
- une erreur de configuration de cellule ;
- une erreur de communication RWS ;

- un process absent du catalogue ;
- un process déclaré mais non implémenté ;
- une erreur survenue pendant l'exécution d'un mouvement.

Après une erreur d'exécution, la routine `TRAJCENTER_ExecuteLoaded` appelle `TRAJCENTER_ProcessAbort` restaure les politiques de configuration de mouvement et propage l'erreur au programme utilisateur.

Une trajectoire ne doit pas être relancée automatiquement sans identifier la cause de l'erreur et sans répéter les contrôles de sécurité nécessaires.

4 Diagnostic et résolution des erreurs

Cette section aide à identifier les incidents les plus fréquents pendant la connexion au contrôleur, le rafraîchissement du store local et le transfert d'une trajectoire.

Le diagnostic doit toujours s'appuyer sur les informations provenant à la fois du PC et du contrôleur. Lorsqu'une erreur survient, relever en priorité :

- les dernières lignes du journal du superviseur Python ;
- la valeur RAPID de `transferError` ;
- la valeur RAPID de `lastErrorCode` ;
- le contenu de `lastError` ;
- l'état de `refreshMetaRequest` et de `sendTrajRequest` ;
- la valeur de `trajReady` ;
- la tâche et le module RAPID ciblés ;
- le nom de la trajectoire demandée.

Les messages écrits dans `lastError` sont volontairement courts, car une chaîne RAPID est limitée à 80 caractères. Le journal Python contient généralement davantage de contexte et doit être consulté en complément.

4.1 Méthode générale de diagnostic

Avant d'analyser un symptôme particulier, appliquer la séquence suivante :

1. vérifier que le contrôleur est accessible depuis le PC ;
2. vérifier que le superviseur est actif ;
3. relever le premier message d'erreur significatif dans le journal ;
4. relever `lastErrorCode` et `lastError` côté RAPID ;
5. vérifier la tâche, le module et le store configurés ;
6. corriger la cause avant de relancer une requête ;
7. rafraîchir les métadonnées si le contenu du store a changé ;
8. redemander la trajectoire après correction.

Pour obtenir davantage de détails, le superviseur peut être lancé temporairement avec le niveau de journalisation `DEBUG` :

```
1 pixi run -e full trajcenter robot supervise \
2   --env-file robot.env \
3   --log-level DEBUG
```

Listing 60 – Activation temporaire des journaux détaillés

Le niveau `DEBUG` est utile pendant le diagnostic, mais le niveau `INFO` reste recommandé pour l'exploitation normale.

4.2 Impossible de se connecter au contrôleur

Symptôme observable.

Le superviseur s'arrête pendant son initialisation ou affiche un message tel que :

```
1 Controller unavailable
```

Listing 61 – Exemple d'erreur de connexion

Le code 503001 peut être associé à cette erreur. Selon la situation, un timeout RWS peut également produire le code 408001 ou 504001.

Causes probables.

- l'adresse définie par `RWS_HOST` est incorrecte ;
- le port `RWS_PORT` est incorrect ;
- le contrôleur est arrêté ou en cours de redémarrage ;
- le PC et le contrôleur ne sont pas sur des réseaux compatibles ;
- un pare-feu bloque la communication ;
- ABB Robot Web Services n'est pas accessible ;
- le délai `RWS_TIMEOUT` est trop court ;
- le protocole ou le port configuré ne correspond pas au contrôleur.

Vérifications à effectuer.

1. vérifier l'adresse IP du contrôleur dans RobotStudio ou sur le FlexPendant ;
2. vérifier les valeurs `RWS_HOST` et `RWS_PORT` dans `robot.env` ;
3. vérifier que le PC peut atteindre l'adresse du contrôleur ;
4. vérifier que le service RWS répond depuis le poste utilisé ;
5. vérifier les règles du pare-feu et la configuration réseau ;
6. consulter le journal détaillé pour distinguer un refus de connexion d'un timeout.

La commande suivante vérifie uniquement que le client ABB est installé :

```
1 pixi run -e full trajcenter robot check
```

Listing 62 – Vérification de l'installation du client ABB

Un résultat positif ne garantit pas que le contrôleur est joignable.

Correction utilisateur.

Corriger l'adresse, le port ou la configuration réseau, puis relancer le superviseur. Si le réseau est lent ou si le contrôleur répond tardivement, augmenter raisonnablement `RWS_TIMEOUT` dans `robot.env` :

```
1 RWS_TIMEOUT=20.0
```

Listing 63 – Exemple d'augmentation du timeout RWS

4.3 Erreur d'authentification

Symptôme observable.

Le superviseur atteint le contrôleur, mais la session RWS est refusée. Le journal affiche généralement :

```
1 RWS authentication refused
```

Listing 64 – Exemple d'erreur d'authentification

Le code associé est 401001.

Causes probables.

- le nom d'utilisateur est incorrect ;
- le mot de passe est incorrect ;
- les guillemets sont absents autour d'un identifiant contenant des espaces ;
- le compte a été modifié ou désactivé ;
- le fichier `.env` utilisé n'est pas celui attendu ;
- une variable d'environnement du système remplace la valeur prévue.

Vérifications à effectuer.

1. vérifier le chemin transmis avec `-env-file` ;
2. vérifier `RWS_USER` et `RWS_PASSWORD` ;
3. placer entre guillemets les valeurs contenant des espaces ;
4. vérifier le compte directement dans la configuration du contrôleur ;
5. rechercher une éventuelle surcharge dans l'environnement du système ou dans la TUI ;
6. redémarrer le superviseur après modification du fichier.

Exemple correct pour un identifiant contenant un espace :

```
1 RWS_USER="Default User"  
2 RWS_PASSWORD=robotics
```

Listing 65 – Écriture d'un identifiant contenant un espace

Correction utilisateur.

Corriger les identifiants ou sélectionner le bon fichier `robot.env`. Ne pas inscrire un mot de passe de production directement dans un script versionné. Après correction, relancer entièrement le superviseur afin de créer une nouvelle session RWS.

4.4 Mastership refusée

Symptôme observable.

La connexion et les lectures RWS fonctionnent, mais l'écriture des données RAPID échoue. Le journal peut afficher :

```
1 Mastership denied by controller
```

Listing 66 – Exemple de refus de Mastership

Le code 403001 indique que la Mastership n'a pas pu être acquise. Le code 403002 indique qu'une écriture RWS a été interdite, par exemple parce que la Mastership n'était pas détenue au moment de l'écriture.

Causes probables.

- un autre client possède déjà la Mastership ;
- RobotStudio conserve la Mastership ;
- une autre application RWS écrit dans la même tâche ;
- les droits du compte ABB sont insuffisants ;
- l'état du contrôleur interdit l'opération ;
- plusieurs superviseurs TrajCenter fonctionnent simultanément ;
- la Mastership est perdue pendant le transfert.

Vérifications à effectuer.

1. vérifier qu'un seul superviseur TrajCenter est actif ;

2. vérifier si RobotStudio ou un autre client détient la Mastership ;
3. vérifier les droits du compte RWS ;
4. vérifier l'état de la tâche RAPID et du contrôleur ;
5. consulter le journal pour connaître le nombre de tentatives effectuées ;
6. vérifier la valeur de `TRAJCENTER_MASTERSHIP_RETRIES`.

Correction utilisateur.

Libérer la Mastership détenue par un autre client, arrêter les superviseurs dupliqués ou utiliser un compte disposant des droits nécessaires. Le nombre de nouvelles tentatives peut être augmenté pour absorber un conflit temporaire :

```
1 TRAJCENTER_MASTERSHIP_RETRIES=5
```

Listing 67 – Configuration des nouvelles tentatives de Mastership

Augmenter cette valeur ne corrige pas un conflit permanent. Si une autre application conserve volontairement la Mastership, il faut coordonner les accès plutôt que multiplier les tentatives.

4.5 Store local introuvable ou vide

Symptôme observable.

Le rafraîchissement des métadonnées échoue, ou `nbTrajAvailable` reste égal à 0. Le journal peut afficher :

```
1 Trajectory store not found
2 No .trajcenter archives found.
```

Listing 68 – Exemples de diagnostic du store

Le code 404004 peut être utilisé lorsque le store est introuvable ou ne peut pas fournir de trajectoire.

Causes probables.

- le chemin du store est incorrect ;
- le répertoire n'existe pas ;
- le superviseur est lancé depuis un autre répertoire de travail ;
- une option `-store` remplace la valeur du fichier `.env` ;
- le répertoire existe mais ne contient aucune archive `.trajcenter` ;
- l'utilisateur exécutant TrajCenter ne possède pas les droits de lecture ;
- les archives se trouvent dans un sous-répertoire non parcouru.

Vérifications à effectuer.

Afficher directement le contenu du store configuré :

```
1 pixi run -e full trajcenter store list \
2 --store trajectory_store
```

Listing 69 – Vérification du contenu du store

Vérifier ensuite :

1. la valeur de `TRAJCENTER_STORE_ROOT` ;
2. l'éventuelle présence de l'option `-store` ;
3. l'existence du répertoire ;
4. la présence de fichiers portant l'extension `.trajcenter` ;

5. les droits de lecture et d'accès ;
6. le chemin du store annoncé dans les journaux au démarrage.

Correction utilisateur.

Créer le répertoire manquant, corriger son chemin ou y placer les archives attendues. Pour éviter les ambiguïtés liées au répertoire de travail, utiliser un chemin absolu lorsque le superviseur est lancé comme service.

Après correction, demander un nouveau rafraîchissement depuis RAPID :

```
TRAJCENTER_RequestMetaRefresh;  
TRAJCENTER_WaitRequestDone 30;
```

Listing 70 – Nouveau rafraîchissement du store

4.6 Trajectoire inconnue

Symptôme observable.

L'appel suivant lève une erreur RAPID locale :

```
TRAJCENTER_RequestTrajByName "production_demo";
```

Listing 71 – Demande susceptible de signaler une trajectoire inconnue

La valeur de `ERRNO` peut correspondre à `ERR_TRAJCENTER_TRAJ_NOT_FOUND`. Une sélection par index peut également produire `ERR_TRAJCENTER_BAD_TRAJ_INDEX` ou le code PC 400001.

Le code 400002 signifie que les métadonnées référencent une archive dont le fichier n'est plus présent au moment du transfert.

Causes probables.

- les métadonnées n'ont pas été rafraîchies ;
- le nom demandé ne correspond pas exactement au nom de l'archive ;
- le nom du fichier est confondu avec le nom présent dans les métadonnées ;
- la casse, un espace ou un caractère diffère ;
- l'archive a été ajoutée après le dernier rafraîchissement ;
- l'archive a été déplacée ou supprimée après le rafraîchissement ;
- l'index demandé n'existe plus dans le catalogue courant.

Vérifications à effectuer.

1. demander un nouveau rafraîchissement des métadonnées ;
2. vérifier que `transferError` vaut `FALSE` après l'attente ;
3. vérifier que `nbTrajAvailable` est supérieur à 0 ;
4. consulter les entrées valides de `trajectories{1..nbTrajAvailable}` ;
5. comparer exactement le nom RAPID avec `trajectories{i}.name` ;
6. vérifier la présence physique de l'archive dans le store.

Correction utilisateur.

Utiliser le nom exact présent dans les métadonnées, replacer l'archive dans le store ou rafraîchir le catalogue après toute modification du répertoire.

Dans un programme de production, préférer la sélection par nom à la sélection par index. L'index peut changer lorsque des archives sont ajoutées, supprimées ou renommées.

4.7 Tool ou workobject absent

Symptôme observable.

Le transfert est refusé alors que l'archive est valide localement. Les codes suivants peuvent être observés :

Code	Signification
400010	Le nom du tool est absent de la trajectoire et aucun tool par défaut n'est autorisé.
400011	Le nom du workobject est absent et aucun workobject par défaut n'est autorisé.
400012	Le nom du tool demandé n'existe pas dans <code>trajTools</code> .
400013	Le nom du workobject demandé n'existe pas dans <code>trajWobjjs</code> .
404002	Le superviseur ne peut pas lire le tableau RAPID <code>trajTools</code> .
404003	Le superviseur ne peut pas lire le tableau RAPID <code>trajWobjjs</code> .

TABLE 4.1 – Codes associés aux tools et workobjects

Une erreur locale telle que `ERR_TRAJCENTER_BAD_TOOL_INDEX`, `ERR_TRAJCENTER_BAD_WOBJ_INDEX`, `ERR_TRAJCENTER_TOOL_NOT_FOUND` ou `ERR_TRAJCENTER_WOBJ_NOT_FOUND` peut également être levée côté RAPID.

Causes probables.

- le nom présent dans l'archive n'est pas déclaré côté robot ;
- le nom diffère par sa casse ou son orthographe ;
- `TRAJCENTER_InitCellConfig` n'a pas été appelée ;
- le tool ou le workobject a été supprimé après le transfert ;
- un nom par défaut est configuré, mais n'existe pas dans les tableaux ;
- les tableaux `trajTools` ou `trajWobjjs` ne sont pas accessibles dans le module ciblé ;
- la trajectoire chargée a été invalidée par une modification de la configuration.

Vérifications à effectuer.

1. relever le nom mentionné dans `lastError` ou dans le journal ;
2. vérifier les noms utilisés dans l'archive ;
3. vérifier les entrées de `trajTools` et `trajWobjjs` ;
4. vérifier `defaultToolName` et `defaultWobjName` ;
5. vérifier les indicateurs `hasDefaultToolName` et `hasDefaultWobjName` ;
6. vérifier que le module ciblé contient bien les symboles attendus ;
7. vérifier que la trajectoire a été rechargée après une modification.

Correction utilisateur.

Déclarer les éléments manquants avec les routines de configuration :

```
TRAJCENTER_UpsertTool
    "productionTool",
    productionTool;

TRAJCENTER_UpsertWobj
    "productionWobj",
    productionWobj;
```

Listing 72 – Ajout d'un tool et d'un workobject manquants

Si l'archive ne fournit pas de nom, configurer explicitement un défaut valide :

```
hasDefaultToolName := TRUE;
defaultToolName := "productionTool";

hasDefaultWobjName := TRUE;
defaultWobjName := "productionWobj";
```

Listing 73 – Configuration de noms par défaut

Après toute modification des tools ou workobjects, redemander la trajectoire. Une trajectoire déjà chargée ne doit pas être exécutée avec une configuration modifiée.

4.8 Trajectoire trop grande

Symptôme observable.

Le transfert est refusé avec le code 400004 et un message tel que :

```
1 Trajectory has too many points
```

Listing 74 – Exemple d'erreur de taille de trajectoire

Dans la configuration de protocole décrite dans ce manuel, une trajectoire ne doit pas dépasser 10000 points.

D'autres limites peuvent également provoquer un refus :

- plus de 256 jeux distincts de paramètres process : code 400018;
- plus de 10 paramètres dans un jeu process : code 400019;
- plus de 256 trajectoires à publier dans le catalogue.

Causes probables.

- le fichier source contient plus de points que prévu;
- des lignes vides ou dupliquées ont été importées comme des points;
- la trajectoire devrait être divisée en plusieurs séquences;
- le nombre de configurations process distinctes est trop élevé;
- les versions Python et RAPID n'utilisent pas les mêmes limites;
- le module RAPID déployé est ancien ou incompatible.

Vérifications à effectuer.

1. inspecter l'archive avec la commande `trajcenter store inspect`;
2. relever le nombre de points indiqué dans les métadonnées;
3. contrôler le fichier source et supprimer les lignes inutiles;
4. vérifier le nombre de jeux de paramètres process;
5. vérifier que le module `TRAJCENTER.sys` déployé correspond à la version Python utilisée;
6. vérifier la valeur RAPID de `maxTrajPointCount`.

Exemple d'inspection :

```
1 pixi run -e full trajcenter store inspect test_basic \
2 --store trajectory_store
```

Listing 75 – Inspection du nombre de points

Correction utilisateur.

Réduire le nombre de points, supprimer les doublons ou découper la trajectoire en plusieurs archives. Réduire également le nombre de jeux de paramètres process lorsque plusieurs points peuvent partager la même configuration.

Les limites du module RAPID et du logiciel Python ne doivent pas être modifiées indépendamment. Une augmentation de capacité constitue une évolution coordonnée du protocole, et non un simple réglage utilisateur.

4.9 Requête RAPID restant à TRUE

Symptôme observable.

Après l'appel d'une routine de demande, l'une des variables suivantes reste à TRUE :

- `refreshMetaRequest` ;
- `sendTrajRequest`.

La progression n'évolue plus, ou la routine d'attente finit par lever `ERR_TRAJCENTER_TIMEOUT`.

Causes probables.

- le superviseur n'est pas démarré ;
- le superviseur s'est arrêté après la réception de la requête ;
- la souscription RWS n'a pas été ouverte correctement ;
- le superviseur cible une autre tâche ou un autre module ;
- la connexion réseau a été interrompue ;
- une erreur empêche le superviseur de remettre le flag à FALSE ;
- la requête a été activée avant la mise en attente du superviseur ;
- un transfert précédent est resté dans un état incohérent.

Vérifications à effectuer.

1. vérifier que le processus du superviseur est toujours actif ;
2. consulter ses dernières lignes de journal ;
3. vérifier la tâche et le module affichés au démarrage ;
4. vérifier l'état de la connexion réseau ;
5. relever `transferError`, `lastErrorCode`, `lastError` et `transferProgress` ;
6. vérifier qu'un seul superviseur est connecté à la cellule ;
7. vérifier qu'aucune seconde requête n'a été envoyée pendant un transfert actif.

Correction utilisateur.

Corriger d'abord la cause initiale, puis redémarrer le superviseur si nécessaire. Ne pas répéter indéfiniment la même demande tant que le flag précédent reste à TRUE.

Si le superviseur ne peut plus traiter la requête, arrêter proprement les clients RWS avant toute intervention sur les variables. La remise manuelle d'un flag à FALSE doit être réservée à la maintenance, lorsque l'absence de transfert actif a été vérifiée. Elle ne doit jamais être réalisée pendant une écriture en cours.

Après rétablissement du superviseur, effectuer d'abord un rafraîchissement des métadonnées, puis redemander la trajectoire.

4.10 Timeout RAPID

Symptôme observable.

Une routine d'attente lève l'erreur locale :

`ERR_TRAJCENTER_TIMEOUT` Cette erreur peut survenir pendant :

- `TRAJCENTER_WaitRequestDone`;
- `TRAJCENTER_WaitTrajectoryReady`.

Le timeout RAPID est une erreur locale. Il ne correspond pas nécessairement à un code PC 408001 ou 408002. Ces derniers décrivent respectivement un timeout de requête RWS ou un timeout global détecté côté Python.

Causes probables.

- le superviseur est arrêté ;
- le délai fourni à la routine est trop court ;
- le transfert d'une grande trajectoire prend plus de temps que prévu ;
- la communication réseau est lente ou instable ;
- la Mastership est temporairement indisponible ;
- la requête reste bloquée à `TRUE` ;
- le contrôleur répond lentement ;
- le superviseur cible une mauvaise tâche ou un mauvais module.

Vérifications à effectuer.

1. vérifier si le flag de requête est toujours à `TRUE` ;
2. vérifier `transferProgress` ;
3. vérifier si le superviseur traite encore la requête ;
4. rechercher un code 408001, 408002, 503001 ou 504001 dans les journaux ;
5. comparer la durée du transfert au timeout RAPID utilisé ;
6. vérifier la stabilité du réseau et la disponibilité de la Mastership ;
7. vérifier la taille de la trajectoire.

Correction utilisateur.

Rétablir le superviseur ou la connexion lorsque le transfert est bloqué. Si le transfert fonctionne mais dépasse régulièrement le délai prévu, augmenter le timeout RAPID :

```
TRAJCENTER_RequestTrajByName "production_demo";  
TRAJCENTER_WaitTrajectoryReady 180;
```

Listing 76 – Augmentation du délai d'attente RAPID

Le timeout doit rester borné. Une attente illimitée masquerait l'arrêt du superviseur ou une panne réseau.

4.11 Superviseur arrêté

Symptôme observable.

Aucune nouvelle requête RAPID n'est traitée. Les flags restent actifs, la progression ne change plus et aucune nouvelle ligne n'apparaît dans le journal du superviseur.

Le processus peut également s'être terminé avec un code non nul.

Causes probables.

- le terminal contenant le superviseur a été fermé ;
- la TUI a arrêté le processus ;
- une erreur non récupérable s'est produite ;
- la connexion au contrôleur a été perdue ;
- le store ou le fichier de configuration est devenu inaccessible ;
- le PC est passé en veille ou a redémarré ;

— le processus a été arrêté par le système d'exploitation.

Vérifications à effectuer.

1. vérifier si le processus TrajCenter est encore actif ;
2. relever son code de retour ;
3. consulter les dernières lignes du journal ;
4. vérifier la disponibilité du contrôleur et du réseau ;
5. vérifier que le fichier `robot.env` existe toujours ;
6. vérifier que le store est accessible ;
7. vérifier que la tâche et le module existent toujours après un redémarrage du contrôleur.

Correction utilisateur.

Corriger l'erreur indiquée dans le journal, puis relancer le superviseur :

```
1 pixi run -e full trajcenter robot supervise
2
3 --env-file robot.env
```

Listing 77 – Redémarrage du superviseur

Après le redémarrage, demander un nouveau rafraîchissement des métadonnées. Ne pas supposer que la requête précédente sera reprise automatiquement.

Pour une exploitation continue, le superviseur peut être lancé par un mécanisme de service adapté au système d'exploitation. Ce mécanisme doit conserver l'accès au fichier de configuration, au store et aux journaux.

4.12 Module ou tâche RAPID incorrects

Symptôme observable.

La connexion RWS fonctionne, mais le superviseur ne peut pas lire ou écrire les symboles TrajCenter. Le journal peut afficher :

```
1 RAPID symbol not found
2 trajTools not found
3 trajWobjjs not found
4 Robot default not found
5 processTypes not found
```

Listing 78 – Exemples d'erreurs de symboles RAPID

Les codes associés peuvent être :

- 404001 : symbole RAPID générique introuvable ;
- 404002 : tableau `trajTools` introuvable ;
- 404003 : tableau `trajWobjjs` introuvable ;
- 404005 : variable de valeur par défaut introuvable ;
- 404006 : catalogue `processTypes` introuvable.

Causes probables.

- `TRAJCENTER_RWS_TASK` ne désigne pas la bonne tâche ;
- `TRAJCENTER_RWS_MODULE` ne correspond pas au nom du module ;
- le module `TRAJCENTER.sys` n'est pas chargé ;
- le module est chargé dans une autre tâche ;
- une option `-task` ou `-module` remplace le fichier `.env` ;
- la TUI transmet une autre configuration de session ;

- le module RAPID déployé appartient à une version incompatible ;
- un symbole attendu a été renommé ou supprimé.

Vérifications à effectuer.

1. vérifier dans RobotStudio la tâche contenant le module ;
2. vérifier le nom exact du module système ;
3. comparer ces valeurs avec le fichier `robot.env` ;
4. vérifier les arguments de la commande de lancement ;
5. vérifier les valeurs affichées dans la TUI ;
6. vérifier que le module contient les variables attendues ;
7. vérifier que le module est chargé sans erreur ;
8. vérifier la compatibilité entre la version Python et la version RAPID.

Exemple de configuration standard :

```
1 TRAJCENTER\_RWS\_TASK=T_ROB1
2 TRAJCENTER\_RWS\_MODULE=TRAJCENTER
```

Listing 79 – Configuration standard de la tâche et du module

Correction utilisateur.

Corriger la tâche ou le module dans `robot.env`, supprimer une surcharge CLI incorrecte ou charger `TRAJCENTER.sys` dans la tâche attendue.

Après correction, redémarrer le superviseur. Une modification de la tâche ou du module n'est pas appliquée rétroactivement à un processus déjà lancé.

4.13 Autres erreurs de validation de trajectoire

Les symptômes précédents couvrent les incidents les plus fréquents. Une trajectoire peut néanmoins être refusée pour une autre erreur de contenu.

Symptôme observable.

Le transfert échoue avec `transferError` à `TRUE` et un code compris entre 400003 et 400019.

Causes probables.

Les principales erreurs restantes sont présentées dans le tableau 4.2.

Vérifications à effectuer.

1. relever le code exact et le message associé ;
2. inspecter l'archive concernée ;
3. vérifier le fichier source utilisé pour la conversion ;
4. vérifier les valeurs par défaut configurées côté robot ;
5. vérifier le catalogue des process ;
6. corriger les données à leur source plutôt que de modifier directement l'archive.

Correction utilisateur.

Corriger le fichier source, convertir de nouveau la trajectoire, remplacer l'archive obtenue dans le store, rafraîchir les métadonnées puis redemander le transfert.

Lorsqu'une erreur concerne une valeur absente, deux solutions sont possibles :

- fournir explicitement la donnée dans la trajectoire ;
- définir et autoriser une valeur par défaut adaptée côté robot.

Code	Problème	Vérification principale
400003	Archive invalide	Vérifier l'intégrité et le format du fichier <code>.trajcenter</code> .
400005	Zone invalide	Vérifier que la zone appartient à la liste prise en charge.
400006	Type de mouvement invalide	Vérifier les valeurs <code>MoveL</code> , <code>MoveJ</code> et <code>MoveC</code> .
400007	Paire <code>MoveC</code> invalide	Vérifier les points circulaires consécutifs et leurs attributs.
400008	Vitesse absente sans défaut	Fournir <code>tcp_speed</code> ou autoriser une vitesse par défaut.
400009	Zone absente sans défaut	Fournir <code>zone_type</code> ou autoriser une zone par défaut.
400014	Vitesse TCP invalide	Vérifier que la vitesse résolue est strictement positive.
400015	Valeur <code>readConfs</code> invalide	Vérifier la représentation booléenne du champ.
400016	<code>robtarger</code> invalide	Vérifier les positions, orientations, configurations et axes externes.
400017	Process inconnu	Vérifier que le process existe dans le catalogue du robot.
400019	Paramètres process invalides	Vérifier les noms, les valeurs numériques et le nombre de paramètres.

TABLE 4.2 – Principales erreurs de validation complémentaires

4.14 Erreurs internes, de sérialisation ou de réponse RWS

Symptôme observable.

Le journal indique un code compris entre 500001 et 502001, par exemple :

- 500001 : erreur interne du client ;
- 500002 : erreur de sérialisation ;
- 500003 : erreur de conversion interne ;
- 502001 : réponse RWS invalide.

Causes probables.

- une valeur Python n'a pas pu être convertie en donnée RAPID ;
- le contrôleur a retourné une réponse inattendue ;
- les structures Python et RAPID sont incompatibles ;
- l'archive contient une combinaison de données non prise en charge ;
- une erreur logicielle non prévue s'est produite.

Vérifications à effectuer.

1. relancer le superviseur avec le niveau `DEBUG` ;
2. conserver le traceback complet du journal Python ;
3. relever la version de TrajCenter ;
4. relever la version du module `TRAJCENTER.sys` ;
5. identifier l'archive et l'étape ayant provoqué l'erreur ;
6. vérifier si l'erreur se reproduit avec une trajectoire minimale connue comme valide.

Correction utilisateur.

Redémarrer le superviseur et tester une seconde fois après vérification des versions. Si l'erreur est reproductible, conserver :

- le journal complet ;
- le code d'erreur ;
- le fichier source ;
- l'archive concernée ;
- les versions Python, TrajCenter, RobotWare et `TRAJCENTER.sys` ;
- la tâche et le module utilisés.

Ces erreurs ne doivent pas être contournées en modifiant manuellement les variables RWS ou les dimensions RAPID.

4.15 Reprise après une erreur

Après correction de la cause, la reprise recommandée est la suivante :

1. vérifier que le superviseur est actif et connecté ;
2. vérifier que les flags de requête précédents ne correspondent plus à une opération active ;
3. demander un rafraîchissement des métadonnées ;
4. attendre la fin de la demande ;
5. contrôler `transferError` ;
6. redemander la trajectoire ;
7. attendre que `trajReady` soit valide ;
8. contrôler `nbLoadedTrajPoints` ;
9. reprendre les vérifications de sécurité avant l'exécution.

Une séquence de reprise RAPID peut être organisée comme suit :

```
TRAJCENTER_RequestMetaRefresh;
TRAJCENTER_WaitRequestDone 30;

IF transferError = TRUE THEN
TPWrite "Metadata refresh failed";
TPWrite "Code:" \Num:=lastErrorCode;
TPWrite lastError;
Stop;
ENDIF

TRAJCENTER_RequestTrajByName "production_demo";
TRAJCENTER_WaitTrajectoryReady 120;

TPWrite "Trajectory loaded";
TPWrite "Point count:" \Num:=nbLoadedTrajPoints;

ERROR
TPWrite "TrajCenter recovery failed";
TPWrite "ERRNO:" \Num:=ERRNO;
TPWrite "Code:" \Num:=lastErrorCode;
TPWrite lastError;
Stop;
```

Listing 80 – Séquence de reprise après correction

Une trajectoire ne doit pas être exécutée uniquement parce que l'erreur précédente a disparu. Les contrôles de sécurité, de configuration et de cohérence doivent être répétés après chaque nouveau transfert.

5 Référence technique et intégration RAPID avancée

Cette section complète le guide d'utilisation par les informations nécessaires aux intégrateurs et aux personnes chargées de maintenir ou d'étendre le module RAPID.

Elle n'est pas nécessaire pour charger et exécuter une trajectoire utilisant le process `NONE`. Elle devient utile pour :

- comprendre le contrat d'échange entre Python et RAPID ;
- comprendre la distinction entre le flux de données et les appels internes du module système ;
- exploiter les paramètres process associés aux points ;
- implémenter un nouveau process dans le dispatcher RAPID ;
- modifier les capacités du module ;
- diagnostiquer une incompatibilité de protocole.

Dans l'architecture actuelle, toutes les trajectoires sont exécutées avec la même routine publique :

`TRAJCENTER_ExecuteLoaded`

Le comportement métier dépend de la valeur de `loadedProcessType`. Le process `NONE` est un cas normal : il utilise le moteur commun de mouvement, mais ses hooks process ne réalisent aucune action.

5.1 Architecture du module système

Le module système constitue l'interface stable entre le superviseur Python et les programmes RAPID de la cellule.

Il remplit deux fonctions distinctes :

- il fournit le contrat de données utilisé par ABB Robot Web Services ;
- il fournit l'API RAPID utilisée par les programmes de cellule.

5.1.1 Rôle et contenu du module

Le fichier `TRAJCENTER.sys` contient :

- les types `RECORD` utilisés pendant les échanges ;
- les dimensions des tableaux RAPID ;
- les codes d'état et d'erreur ;
- les types de mouvement pris en charge ;
- les identifiants et le catalogue des process exécutables ;
- les tools et workobjects exposés au superviseur ;
- les variables persistantes et runtime accessibles par RWS ;
- les valeurs par défaut définies par la cellule ;

- l'API RAPID publique ;
- les routines de support ;
- les interfaces process ;
- le dispatcher process ;
- le moteur commun de mouvement.

Sa structure générale peut être résumée ainsi :

```
MODULE TRAJCENTER(SYSMODULE)
  ! Types RECORD
  ! Limites globales
  ! Codes d'etat et d'erreur
  ! Types de mouvement
  ! Identifiants et catalogue process
  ! Configuration cellule
  ! Variables d'echange RWS
  ! Valeurs par default robot
  ! API publique TRAJCENTER_*
  ! Acces aux parametres process
  ! Implementations process
  ! Dispatcher process
  ! Moteur commun de mouvement
  ! API publique d'execution
ENDMODULE
```

Listing 81 – Structure générale du module système TRAJCENTER

Les différentes parties du module n'ont pas la même politique de modification :

5.1.2 Flux fonctionnel des données

La figure 5.1 présente le flux de données complet, depuis la conversion d'un fichier source jusqu'à la publication du payload dans le contrôleur.

Partie	Contenu	Politique de modification
Types et limites	RECORD échangés, dimensions des tableaux et types de mouvement.	À modifier uniquement avec une évolution coordonnée du protocole Python/RAPID.
Codes d'état et d'erreur	États du transfert, erreurs PC/RWS et erreurs locales RAPID.	À maintenir en cohérence avec le logiciel Python, les tests et la documentation.
Catalogue process	Identifiants et liste des process déclarés exécutables par le contrôleur.	Ne publier un process que lorsque son implémentation RAPID est opérationnelle.
Configuration cellule	Tools et workobjects exposés au superviseur.	Modifiable pendant l'intégration et la maintenance de la cellule.
Variables RWS	Requêtes, états, métadonnées, points, type de process chargé et paramètres process.	Ne pas modifier indépendamment du client Python.
Valeurs par défaut	Vitesse, zone, mouvement, tool, workobject et lecture des configurations.	À régler pendant la mise en service de la cellule.
API RAPID	Initialisation, configuration, requêtes, attentes, accès aux données et exécution.	Interface destinée aux programmes de cellule.
Routines process	Hooks <code>Begin</code> , <code>BeforeMotion</code> , <code>AfterMotion</code> , <code>End</code> et <code>Abort</code> .	Point d'extension destiné à l'implémentation des process métier.
Dispatcher et moteur commun	Sélection du process et exécution des mouvements <code>MoveL</code> , <code>MoveJ</code> et <code>MoveC</code> .	À maintenir comme chemin d'exécution commun.

TABLE 5.1 – Organisation technique du module système

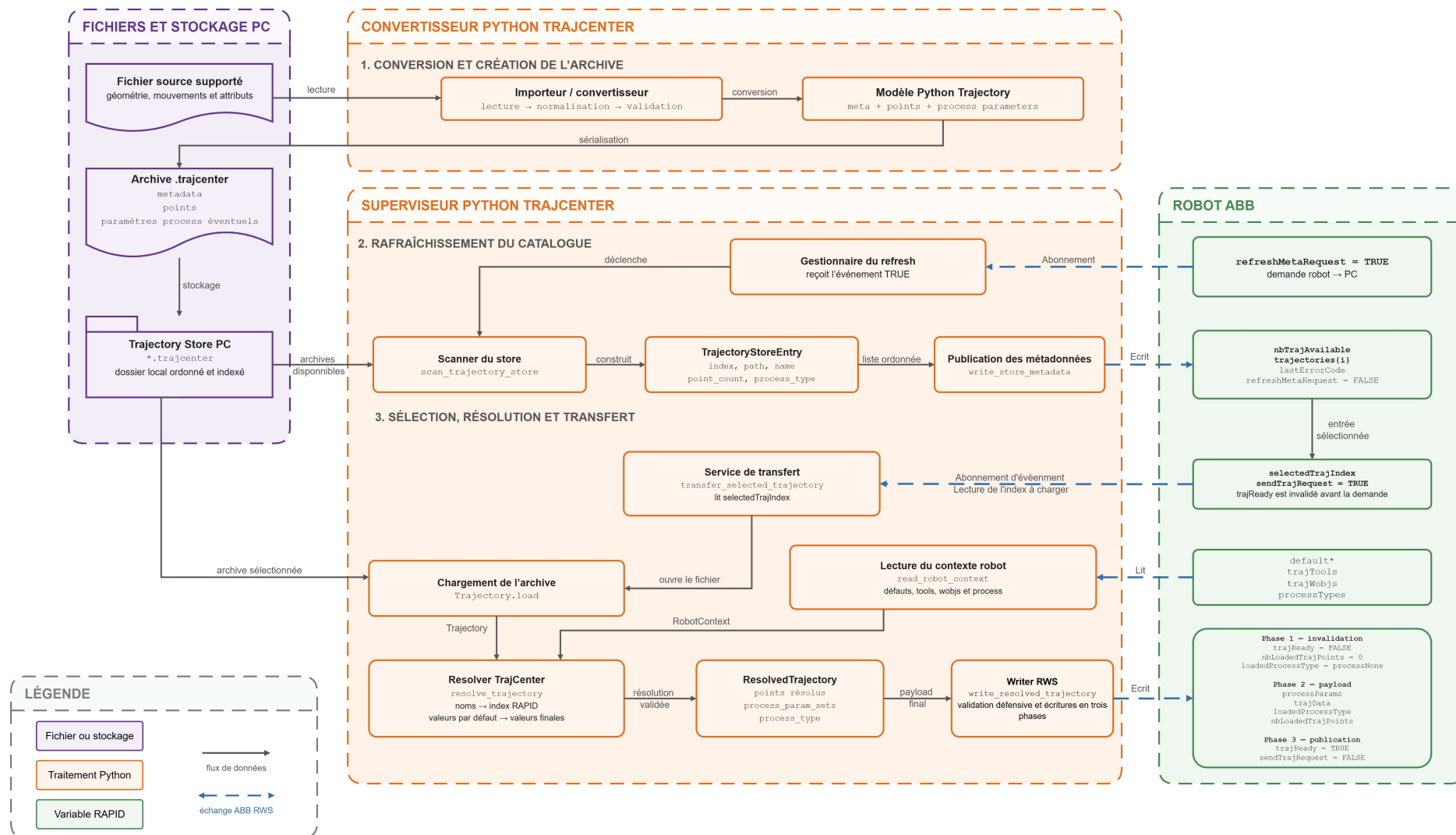


FIGURE 5.1 – Flux de données TrajCenter entre le store PC, Python et RAPID

Cette figure représente un **flux de données**. Elle ne doit pas être interprétée comme un graphe d'appels de fonctions.

Le cycle général est le suivant :

1. un fichier source est converti en archive `.trajcenter` ;
2. l'archive est placée dans le store local ;
3. le superviseur scanne le store et construit le catalogue ;
4. le catalogue est publié dans `nbTrajAvailable` et `trajectories` ;
5. le robot sélectionne une entrée du catalogue ;
6. Python charge l'archive effectivement sélectionnée ;
7. Python vérifie sa cohérence avec le snapshot du catalogue ;
8. Python lit les defaults, les tools, les workobjects et le catalogue process du contrôleur ;
9. le resolver produit une trajectoire entièrement résolue ;
10. le writer valide le payload puis l'écrit dans RAPID.

Les flèches bleues entre Python et RAPID représentent les échanges utilisant ABB Robot Web Services. RWS est un canal de communication et non un composant physique intermédiaire.

Le transfert d'une trajectoire suit un protocole en trois phases :

1. **invalidation** : `trajReady` passe à `FALSE`, `nbLoadedTrajPoints` passe à 0 et `loadedProcessType` revient à `processNone` ;
2. **écriture du payload** : Python écrit `processParams`, `trajData`, `loadedProcessType` et `nbLoadedTrajPoints` ;
3. **publication** : Python met `trajReady` à `TRUE`, puis remet `sendTrajRequest` à `FALSE`.

Ainsi, les anciennes valeurs éventuellement présentes dans les tableaux ne sont jamais considérées comme valides tant que `trajReady` ne vaut pas `TRUE`.

Le programme RAPID utilisateur ne doit pas écrire directement dans :

- `trajectories` ;
- `trajData` ;
- `processParams` ;
- `loadedProcessType` ;
- `nbLoadedTrajPoints` ;
- les variables d'état gérées par le protocole.

Ces données sont alimentées par le superviseur sous Mastership ou modifiées par les routines internes du module.

5.1.3 Validité du payload chargé

La validité du payload est représentée par trois variables principales :

- `trajReady` indique si le payload complet est publiable ;
- `nbLoadedTrajPoints` indique la plage valide dans `trajData` ;
- `loadedProcessType` indique le process associé au payload effectivement chargé.

La routine interne `TRAJCENTER_InvalidateLoaded` applique l'état d'invalidation suivant :

```
trajReady := FALSE;
nbLoadedTrajPoints := 0;
loadedProcessType := processNone;
```

Listing 82 – État d'invalidation du payload chargé

Les tableaux `trajData` et `processParams` ne sont pas nécessairement effacés. Leurs anciennes valeurs deviennent simplement inaccessibles, car leurs compteurs et leur indicateur de validité ne permettent plus de les utiliser.

Le payload est invalidé notamment :

- avant une nouvelle demande de trajectoire ;
- lorsqu'un tool est ajouté ou modifié ;
- lorsqu'un workobject est ajouté ou modifié ;
- lorsqu'un tool ou un workobject est supprimé ;
- lorsque la configuration cellule est vidée ;
- lorsqu'un transfert échoue avant sa publication complète.

La routine `TRAJCENTER_RequestMetaRefresh` n'invalidé pas le payload chargé. Un rafraîchissement du catalogue peut donc être réalisé sans rendre inexécutable une trajectoire déjà publiée, tant que la configuration de cellule associée à cette trajectoire n'est pas modifiée.

5.1.4 Compatibilité des versions

Les dimensions et structures du module RAPID doivent être compatibles avec le profil de protocole pris en charge par la version Python installée.

Le fichier interne Python `trajcenter/robot/constants.py` n'est pas un fichier de configuration utilisateur. Il décrit les conventions et capacités attendues par cette version du logiciel.

Une modification portant sur les éléments suivants nécessite une évolution coordonnée :

- dimensions des tableaux ;
- types RECORD ;
- ordre des champs dans les RECORD ;
- noms ou emplacement des variables RWS ;
- codes d'état et d'erreur ;
- représentation des types de mouvement ;
- identifiants des types de process ;
- structure des paramètres process.

Dans la version décrite par ce manuel, la capacité maximale d'une trajectoire est fixée à 10000 points côté Python et côté RAPID. Les constantes correspondantes doivent rester strictement alignées.

Toutes les dimensions du module RAPID ne sont pas nécessairement comparées automatiquement avec le profil Python au démarrage. Le module `TRAJCENTER.sys` et le logiciel Python doivent donc être déployés dans des versions compatibles.

5.2 API RAPID publique

Le tableau 5.2 résume les principales routines destinées aux programmes de cellule.

La routine `TRAJCENTER_ExecuteLoaded` est l'unique point d'entrée recommandé pour l'exécution. Il n'existe plus d'API d'exécution séparée pour les trajectoires sans process.

Une trajectoire sans process correspond au cas suivant :

```
loadedProcessType = processNone
```

Listing 83 – Représentation d'une trajectoire sans process

Les signatures exactes doivent toujours être vérifiées dans le module `TRAJCENTER.sys` correspondant à la version installée.

5.3 Intégration des paramètres process

Les paramètres process sont transférés dans le tableau `processParams`. Chaque point peut référencer un jeu de paramètres au moyen de son champ `processParamIndex`.

La convention est la suivante :

- `processParamIndex = 0` signifie qu'aucun jeu de paramètres n'est associé au point ;
- une valeur comprise entre 1 et `maxProcessParamSetCount` référence une ligne de `processParams` ;
- dans un jeu, un nom vide indique un slot inutilisé ;
- les paramètres process transportés par le protocole sont numériques.

5.3.1 Catalogue des process exécutables

Le tableau `processTypes` représente les process réellement exécutables par le contrôleur. Le superviseur Python lit ce catalogue avant d'accepter une trajectoire.

Dans la version actuelle du module :

```
CONST num processTypeCount := 1;

VAR trajCenterProcessType processTypes{processTypeCount}:=[
    [processNone, "NONE"]
];
```

Listing 84 – Catalogue process actuellement publié

Les identifiants `processAcf`, `processAak` et `processPushcorp` sont réservés dans le module, mais ils ne sont pas publiés dans le catalogue. Leurs routines `Begin` lèvent encore `ERR_TRAJCENTER_PROCESS_NOT_IMPLEMENTED`.

Une trajectoire utilisant l'un de ces process doit donc être refusée par le superviseur tant que son implémentation `RAPID` n'est pas opérationnelle et que le process n'a pas été ajouté à `processTypes`.

5.3.2 Accès contrôlé aux paramètres process

Les routines process doivent utiliser `TRAJCENTER_PointHasProcessParams` et `TRAJCENTER_GetProcessParam` plutôt que lire directement les tableaux internes.

L'exemple suivant recherche un paramètre numérique associé au point courant :

```
VAR bool found;
VAR num processValue;

TRAJCENTER_GetProcessParam
    pointIndex,
    "example_parameter",
    found,
    processValue;

IF found = FALSE THEN
    RAISE ERR_TJC_BAD_PROCESS_PARAM_INDEX;
ENDIF
```

Listing 85 – Lecture d'un paramètre process pour un point

Cette API centralise :

- le contrôle de l'index du point ;
- le contrôle de l'index du jeu de paramètres ;
- la recherche du paramètre par son nom ;
- la distinction entre paramètre absent et valeur numérique nulle.

5.3.3 Implémentation d'un process métier

L'ajout d'un process ne nécessite pas de créer une nouvelle routine globale d'exécution. Le moteur de mouvement est déjà fourni par `TRAJCENTER_ExecuteOneMotion`.

Chaque process doit implémenter cinq hooks :

Begin initialise le process avant le premier mouvement ;

BeforeMotion applique les paramètres avant l'instruction de mouvement ;

AfterMotion réalise les actions nécessaires après l'instruction de mouvement ;

End termine normalement le process ;

Abort remet le process dans un état sûr lorsqu'une erreur survient.

Pour ajouter un process, appliquer la séquence suivante :

1. définir ou confirmer son identifiant dans la section 5 du module ;
2. implémenter ses cinq routines process dans la section 9.10 ;
3. ajouter les appels correspondants dans chacun des dispatchers de la section 9.11 ;
4. ajouter le process dans le tableau `processTypes` ;
5. incrémenter `processTypeCount` ;
6. mettre à jour les constantes et validations Python ;
7. ajouter les tests correspondants ;
8. mettre à jour la documentation.

Un process ne doit être ajouté à `processTypes` qu'une fois son implémentation RAPID réellement utilisable.

Pour les hooks **BeforeMotion** et **AfterMotion**, l'index fourni correspond au premier point du mouvement courant. Pour un **MoveC**, il désigne le point de passage ; le point d'arrivée est situé à l'index suivant.

Les hooks sont appelés une fois par instruction de mouvement, et non une fois par entrée du tableau `trajData`. Un **MoveC** consomme donc deux entrées, mais provoque un seul appel à chaque hook.

Une action réalisée dans **AfterMotion** n'est pas nécessairement synchronisée avec le passage physique exact au point lorsque la zone n'est pas **fine**. Un procédé nécessitant une synchronisation précise doit utiliser une stratégie ABB adaptée, par exemple des triggers, des instructions process spécialisées ou un moteur dédié explicitement conçu pour cette synchronisation.

5.4 Routines internes et graphe d'appels

Certaines routines RAPID sont présentes dans le module pour factoriser les opérations internes. Elles ne font pas toute partie du parcours d'utilisation courant.

Le fait qu'une routine soit techniquement accessible depuis un programme RAPID ne signifie pas qu'elle constitue une API publique stable. Les programmes de cellule doivent privilégier les points d'entrée documentés.

5.4.1 Principales routines de support

Les routines de support prennent notamment en charge :

- l’invalidation du payload chargé ;
- la recherche des tools et workobjects ;
- la recherche d’une trajectoire par son nom ;
- la construction des données de vitesse ;
- la conversion des zones ;
- le chargement des caches d’exécution tool et workobject ;
- l’application de la politique `readConfs` ;
- l’accès contrôlé aux paramètres process ;
- la validation et l’exécution d’un mouvement ;
- le dispatch des appels process.

Les principales routines concernées sont :

- `TRAJCENTER_InvalidatedLoaded` ;
- `TRAJCENTER_FindToolIndex` ;
- `TRAJCENTER_FindWobjIndex` ;
- `TRAJCENTER_FindTrajectoryIndex` ;
- `TRAJCENTER_MakeSpeed` ;
- `TRAJCENTER_MakeZone` ;
- `TRAJCENTER_LoadExecTool` ;
- `TRAJCENTER_LoadExecWobj` ;
- `TRAJCENTER_ApplyReadConfs` ;
- `TRAJCENTER_ProcessBegin` ;
- `TRAJCENTER_ProcessBeforeMotion` ;
- `TRAJCENTER_ProcessAfterMotion` ;
- `TRAJCENTER_ProcessEnd` ;
- `TRAJCENTER_ProcessAbort` ;
- `TRAJCENTER_ExecuteOneMotion`.

5.4.2 Moteur commun de mouvement

La routine `TRAJCENTER_ExecuteOneMotion` constitue l’unique implémentation générique des mouvements.

Pour chaque mouvement, elle :

1. lit le point courant dans `trajData` ;
2. construit le `speeddata` ;
3. convertit la zone numérique en `zonedata` ;
4. charge le tool dans `trajExecTool` ;
5. charge le workobject dans `trajExecWobj` ;
6. applique la politique `readConfs` ;
7. appelle le hook `ProcessBeforeMotion` ;
8. exécute `MoveL`, `MoveJ` ou `MoveC` ;
9. appelle le hook `ProcessAfterMotion` ;
10. avance l’index d’un ou de deux points.

Pour un mouvement circulaire, la routine vérifie que les deux entrées consécutives :

- sont toutes les deux de type `moveTypeC` ;
- utilisent la même vitesse TCP ;
- utilisent le même tool ;
- utilisent le même workobject ;
- utilisent la même zone ;
- utilisent la même politique `readConfs` ;
- référencent le même jeu de paramètres process.

5.4.3 Graphe d'appels

Routine	Arguments	Utilisation
TRAJCENTER_InitErrors	Aucun	Initialise les erreurs RAPID locales.
TRAJCENTER_InitCellConfig	Aucun	Ajoute ou met à jour <code>tool0</code> et <code>wobj0</code> .
TRAJCENTER_UpsertTool	string, tooldata	Ajoute ou met à jour un tool et invalide le payload chargé.
TRAJCENTER_UpsertWobj	string, wobjdata	Ajoute ou met à jour un workobject et invalide le payload chargé.
TRAJCENTER_DeleteTool	string	Supprime un tool et invalide le payload chargé.
TRAJCENTER_DeleteWobj	string	Supprime un workobject et invalide le payload chargé.
TRAJCENTER_ClearTools	Aucun	Vide les tools exposés et invalide le payload chargé.
TRAJCENTER_ClearWobjs	Aucun	Vide les workobjects exposés et invalide le payload chargé.
TRAJCENTER_ClearCellConfig	Aucun	Vide les tools et workobjects exposés.
TRAJCENTER_RequestMetaRefresh	Aucun	Demande le rafraîchissement du catalogue sans invalider le payload chargé.
TRAJCENTER_WaitRequestDone	num timeoutSec	Attend la fin d'une requête de rafraîchissement.
TRAJCENTER_RequestTrajectory	num trajIndex	Invalide le payload courant et demande une trajectoire par index.
TRAJCENTER_RequestTrajByName	string trajName	Recherche une trajectoire par nom puis la demande par index.
TRAJCENTER_WaitTrajectoryReady	num timeoutSec	Attend la publication complète du payload.
TRAJCENTER_GetLoadedPointCount	Aucun	Retourne le nombre de points valides.
TRAJCENTER_GetPoint	num, VAR trajCenterPointData	Copie un point chargé dans une variable utilisateur.
TRAJCENTER_GetTool	num, VAR tooldata	Copie un tool dans une variable utilisateur.
TRAJCENTER_GetWobj	num, VAR wobjdata	Copie un workobject dans une variable utilisateur.
TRAJCENTER_PointHasProcessParameters	num pointIndex	Indique si un point référence un jeu de paramètres process.
TRAJCENTER_GetProcessParam	Point, nom et variables de sortie	Recherche un paramètre process numérique associé à un point.
TRAJCENTER_ExecuteLoaded	Trois valeurs num	Exécute le payload chargé avec le process indiqué par <code>loadedProcessType</code> .

TABLE 5.2 – Principales routines publiques de TrajCenter

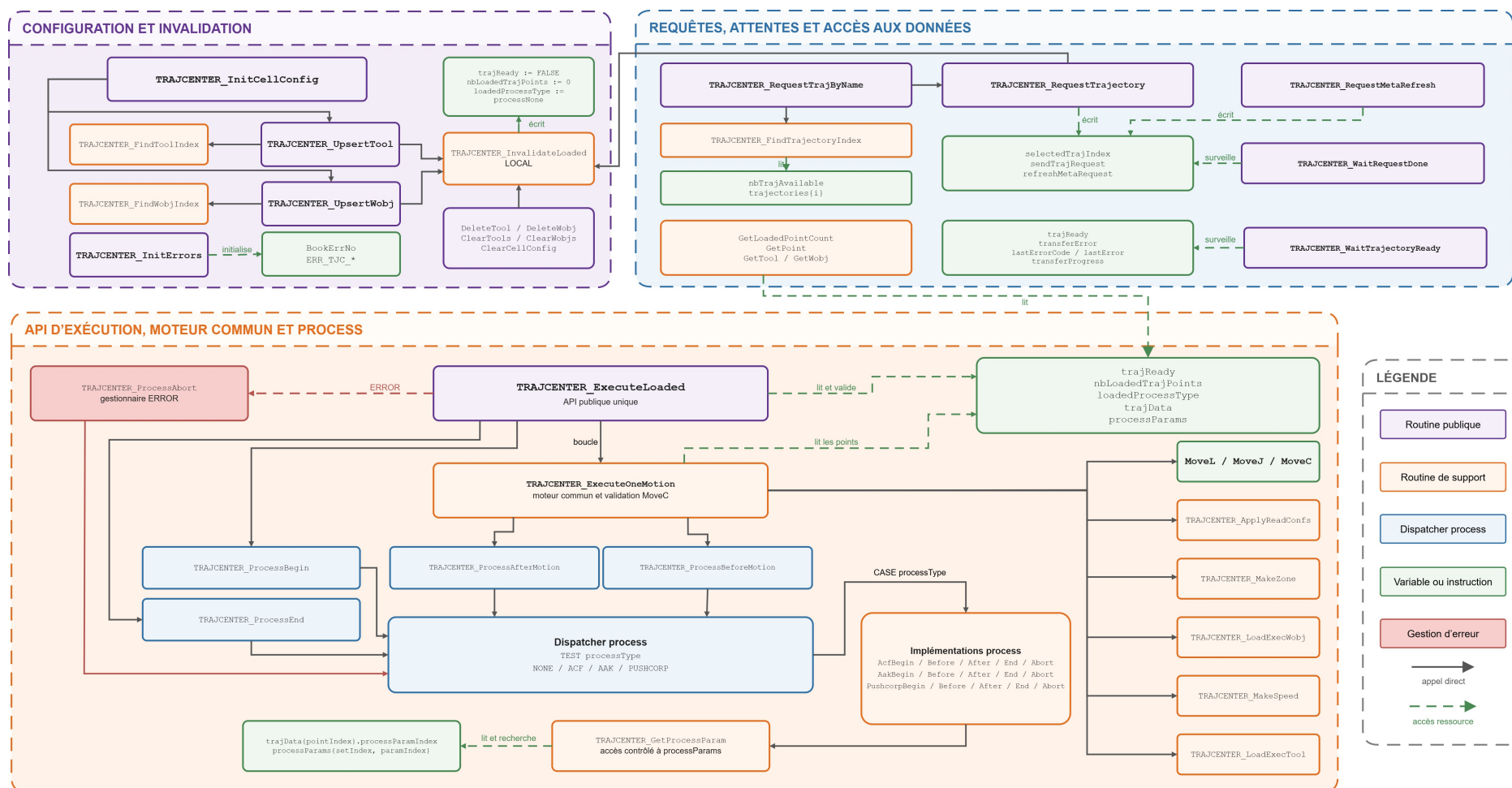


FIGURE 5.2 – Graphe d'appels et interdépendances du module système RAPID

La figure 5.2 présente une vue simplifiée des appels et interdépendances internes du module système RAPID. Dans ce graphe :

- une flèche pleine représente un appel direct entre routines RAPID ;
- une flèche verte en pointillés représente l'accès à une variable ou à une instruction RAPID ;
- une flèche rouge représente le chemin de gestion d'erreur ;
- une liaison d'extension indique une dépendance prévue pour une implémentation process.

La routine `TRAJCENTER_ExecuteLoaded` organise l'exécution globale :

1. elle vérifie `trajReady` et `nbLoadedTrajPoints` ;
2. elle copie `loadedProcessType` dans une variable locale ;
3. elle appelle `TRAJCENTER_ProcessBegin` ;
4. elle appelle `TRAJCENTER_ExecuteOneMotion` jusqu'à la fin du payload ;
5. elle appelle `TRAJCENTER_ProcessEnd` ;
6. elle restaure `ConfL` et `ConfJ` à `\On`.

Si une erreur survient, son gestionnaire `ERROR` :

1. appelle `TRAJCENTER_ProcessAbort` ;
2. restaure `ConfL` et `ConfJ` ;
3. propage l'erreur originale avec `RAISE`.

Cette structure garantit que le process `NONE` et les futurs process métier empruntent le même moteur de mouvement et la même stratégie de remise en état.

5.5 Règles de modification du module

Une modification locale de la configuration de la cellule n'a pas le même impact qu'une modification du protocole ou du moteur d'exécution.

5.5.1 Modifications propres à la cellule

Les opérations suivantes relèvent normalement de l'intégration de la cellule :

- ajouter ou mettre à jour un tool ;
- ajouter ou mettre à jour un workobject ;
- choisir les valeurs par défaut ;
- configurer les équipements associés à un process ;
- compléter les hooks d'un process déjà réservé ;
- associer des sorties, instructions ou données métier aux hooks process.

Ces opérations ne doivent pas modifier la structure des variables échangées avec Python.

Toute modification d'un tool ou d'un workobject invalide le payload chargé. La trajectoire doit alors être redemandée avant toute nouvelle exécution.

5.5.2 Ajout ou activation d'un process

L'activation d'un process doit être réalisée de manière coordonnée :

1. confirmer son identifiant numérique ;
2. implémenter ses cinq hooks RAPID ;
3. compléter les cinq dispatchers ;

4. vérifier que son hook `Abort` ne masque pas l'erreur originale ;
5. ajouter le process à `processTypes` ;
6. incrémenter `processTypeCount` ;
7. mettre à jour les constantes et validations Python ;
8. ajouter des trajectoires et tests représentatifs ;
9. mettre à jour la documentation.

La simple présence d'une constante telle que `processAcf` ne signifie pas que le process est disponible. Seules les entrées valides de `processTypes` constituent le catalogue publié au superviseur.

5.5.3 Modifications du protocole

Les opérations suivantes constituent une évolution du protocole :

- changer les dimensions des tableaux ;
- modifier un type `RECORD` échangé ;
- modifier l'ordre des champs d'un `RECORD` ;
- ajouter, supprimer ou renommer une variable `RWS` ;
- modifier la représentation d'un type de mouvement ;
- modifier les identifiants des process ;
- modifier les codes d'état ou d'erreur ;
- modifier la structure des paramètres process ;
- modifier le protocole d'invalidation et de publication du payload.

Elles doivent être réalisées simultanément :

1. dans le module `RAPID` ;
2. dans le client Python ;
3. dans les constantes et validations Python ;
4. dans les tests unitaires et d'intégration ;
5. dans les démonstrations `RAPID` ;
6. dans la documentation utilisateur et technique ;
7. dans la gestion de version du module et du logiciel.

Il ne faut pas utiliser le fichier `.env` pour contourner une incompatibilité de dimensions ou de structures. Le fichier `.env` configure la connexion et le superviseur ; il ne redéfinit pas le protocole.